



Facultad de Ingeniería

Carrera De Ingeniería De Sistemas E Informática

Tesis

**“Sistema De Intrusión De Trafico De Red Basado En Machine Learning Para La
Protección De Datos En La Empresa, Lima, 2024”**

Autores

Figueroa Dextre Betty Sonaira (U18204825)

Cañihua Ccapa Jesús (U18217407)

para optar el Título Profesional de Ingeniero de Sistemas e Informática

Asesor

Dr. Ponte Quiñones Elvis Jerson

Lima - Perú

Marzo del 2024

14% Similitud general

El total combinado de todas las coincidencias, incluidas las fuentes superpuestas, para ca...

Filtrado desde el informe

► Bibliografía

Fuentes principales

- 12%  Fuentes de Internet
- 3%  Publicaciones
- 5%  Trabajos entregados (trabajos del estudiante)

RESUMEN

La digitalización de los procesos empresariales ha incrementado los riesgos de ciberseguridad, generando la necesidad de sistemas avanzados que protejan la información corporativa. En este contexto, la presente tesis desarrolla un sistema de detección de intrusiones en el tráfico de red basado en machine learning, con el propósito de mejorar la seguridad informática en un entorno empresarial en Lima, 2024.

El estudio parte de la identificación de las deficiencias en los sistemas tradicionales, los cuales presentan limitaciones frente a ataques como el malware, amenazas de día cero y ataques de denegación de servicio (DoS). Para abordar este problema, se implementaron algoritmos de machine learning, tales como Random Forest y Regresión Logística, con el fin de analizar patrones en el tráfico de red e identificar anomalías que representen amenazas.

Metodológicamente, la investigación adopta un enfoque cuantitativo- experimental y aplica la metodología ágil SCRUM. Se utilizó el conjunto de datos CICIDS2017 para entrenar y evaluar los modelos en un entorno de prueba, midiendo su desempeño mediante métricas como precisión, recall y F1-score.

Los resultados demostraron que la incorporación de técnicas de machine learning mejora la detección de amenazas, reduciendo de manera notable los falsos positivos en comparación con los sistemas tradicionales. De forma destacada, se evidenció que el modelo basado en Random Forest mostró el mejor desempeño, alcanzando una precisión del 95% y obteniendo valores de precisión, recall y F1-score de 0.94, 0.93 y 0.93, la Regresión Logística presentó una mayor velocidad de procesamiento.

En conclusión, este trabajo respalda la viabilidad y eficacia de aplicar modelos de machine learning especialmente Random Forest para optimizar la detección de intrusiones y mejorar la respuesta ante amenazas cibernéticas, consolidando una herramienta robusta para la seguridad en entornos empresariales modernos.

Palabras clave: Ciberseguridad, machine learning, detección de intrusiones, Random Forest, Regresión Logística, CICIDS2017, protección de datos.

ABSTRACT

The digitalization of business processes has increased cybersecurity risks, creating the need for advanced systems to protect corporate information. In this context, the present thesis develops a network traffic intrusion detection system based on machine learning, with the goal of enhancing cybersecurity in a business environment in Lima, 2024.

The study begins by identifying the shortcomings of traditional systems, which present limitations when facing attacks such as malware, zero-day threats, and denial-of-service (DoS) attacks. To address this issue, machine learning algorithms such as Random Forest and Logistic Regression were implemented to analyze patterns in network traffic and detect anomalies that represent potential threats.

Methodologically, the research adopts a quantitative-experimental approach and applies the agile SCRUM methodology. The CICIDS2017 dataset was used to train and evaluate the models in a test environment, measuring their performance using metrics such as accuracy, recall, and F1-score.

The results demonstrated that incorporating machine learning techniques improves threat detection, significantly reducing false positives compared to traditional systems. Notably, the Random Forest-based model showed the best performance, achieving an accuracy of 95% and obtaining precision, recall, and F1-score values of 0.94, 0.93, and 0.93, respectively. Logistic Regression, on the other hand, demonstrated faster processing speed.

In conclusion, this work supports the feasibility and effectiveness of applying machine learning models especially Random Forest to optimize intrusion detection and improve response to cyber threats, establishing a robust tool for security in modern business environments.

Keywords: Cybersecurity, machine learning, intrusion detection, Random Forest, Logistic Regression, CICIDS2017, data protection.

INDICE GENERAL

RESUMEN	III
ABSTRACT	IV
INDICE DE TABLAS	VIII
INDICE DE FIGURAS	IX
ANEXO	X
CAPITULO I: INTRODUCCIÓN	1
CAPITULO II: PLANTEAMIENTO DEL PROBLEMA	2
2.1. Descripción del problema	2
2.2. Problema general	3
2.3. Problemas específicos	3
CAPÍTULO III: DETERMINACIÓN DE OBJETIVOS	4
3.1. Objetivos generales	4
3.2. Objetivos específicos	4
CAPÍTULO IV: HIPOTESIS Y VARIABLES	5
4.1. Hipótesis generales	5
4.2. Hipótesis específicas	5
4.3. Variables	5
4.4. Justificación	6
4.4.1. Teórica	6
4.4.2. Práctica	6
4.4.3. Metodológica	6
CAPÍTULO V: MARCO TEORICO Y ANTECEDENTES	9
5.1. Marco Teórico	9
5.1.1. Protección de Datos en Empresas	9
5.1.2. Seguridad de la Información	10
5.1.3. Pilares de la Seguridad de la Información	10
5.1.4. Amenazas comunes	11
5.1.5. Sistema de Gestión de Seguridad de la Información (SGSI)	12
5.1.6. Tráfico de red	13
5.1.7. Sistema de Detección de Intrusiones (IDS)	15
5.1.8. Machine Learning en Ciberseguridad	17
5.1.9. Dataset	19

5.1.10.	Vulnerabilidades	21
5.2.	Estado del Arte	21
CAPITULO VI: METODOLOGÍA DE LA INVESTIGACIÓN.....		24
6.1.	Diseño de la investigación	24
6.2.	Tipo de la investigación.....	24
6.3.	Alcance.....	24
6.4.	Enfoque	25
6.5.	Población y muestra	25
6.6.	Instrumento de Medición y Técnica.....	25
6.7.	Recolección del Conjunto de Datos y Variables Utilizadas	26
6.8.	Proceso Experimental de Entrenamiento y Prueba de los Modelos.....	26
6.9.	Métricas de Selección.....	28
6.10.	Metodología de medición	29
6.11.	Selección de la metodología.....	30
6.12.	Desarrollo de la metodología.....	33
CAPITULO VII: DESARROLLO DE RESULTADOS Y DISCUSIÓN		39
7.1.	Resultados.....	39
7.2.	Pruebas con Wireshark.....	39
7.2.1.	Captura de tráfico de red con Wireshark	39
7.2.2.	Conversaciones de Red.....	41
7.2.3.	Análisis de Paquetes con Filtros	42
7.2.4.	Detección de Ataques DoS/DDoS.....	43
7.2.5.	Detección de Port Scanning	44
7.2.6.	Tráfico de DNS Sospechoso.....	45
7.3.	Pruebas de algoritmos de Machine Learning	46
7.3.1.	Dataset de entrenamiento.....	46
7.3.2.	Métricas de la Matriz de Confusión (Random Forest).....	47
7.3.3.	Reporte de Desempeño del Modelo (Random Forest)	48
7.3.4.	Distribución de Predicciones (Random Forest)	49
7.4.	Métricas de la Matriz de Confusión (Regresión Logística)	50
7.4.1.	Entrenamiento y Validación	50
7.5.	Comparativa de detección de tráfico de red entre Random Forest y Regresión logística.....	53
7.5.1.	Estadísticas de precisión.....	54
CAPITULO VIII: DISCUSIÓN		57
CAPITULO IX: CONCLUSIONES Y RECOMENDACIONES.....		60

9.1. Conclusiones	60
9.2. Recomendaciones	61
BLIBLIOGRAFIA.....	62
ANEXO.....	68

INDICE DE TABLAS

TABLA 1	Variable dependiente e independiente	5
TABLA 2	Ventajas y Desventajas de la Clasificación del tráfico de red.	15
TABLA 3	Diferentes tipos de machine learning que se aplican en ciberseguridad.	17
TABLA 4	Comparación de características de la Metodologías ágiles vs Metodologías tradiciones.	32
TABLA 5	Fase 1. Inicio	33
TABLA 6	Fase 2. Planificación y Estimación.....	34
TABLA 7	Fase 3. Diseño.	35
TABLA 8	Fase 4. Implementación.....	36
TABLA 9	Fase 5. Revisión y Retrospectiva.	37
TABLA 10	Fase 6. Lanzamiento y Pruebas.	38
TABLA 11	<i>Comparación de los algoritmos Random forest y Regresión logística</i>	<i>53</i>
TABLA 12	<i>Estadísticas de precisión</i>	<i>54</i>
TABLA 13	<i>Características de las máquinas virtuales instaladas</i>	<i>77</i>

INDICE DE FIGURAS

FIGURA 1 Arquitectura de protección de la información empresarial.....	10
FIGURA 2 Pilares de seguridad de información	11
FIGURA 3 Ciclo de mejora continua (PDCA).....	13
FIGURA 4 Clasificación de IDS	16
FIGURA 5 Arquitectura de análisis de datos.....	21
FIGURA 6 Procesos SCRUM.....	32
FIGURA 7 Captura de tráfico de red con Wireshark – 1.....	40
FIGURA 8 Captura de tráfico de red con Wireshark – 2.....	40
FIGURA 9 Captura de tráfico de red con Wireshark – 3.....	41
FIGURA 10 Captura de tráfico de red con Wireshark – 4.....	41
FIGURA 11 Detección de Tráfico con Wireshark (Conversaciones de Red)	42
FIGURA 12 Detección de Tráfico con Wireshark (Análisis de Paquetes con Filtros) – 1.	43
FIGURA 13 Detección de Tráfico con Wireshark (Análisis de Paquetes con Filtros) – 2.	43
FIGURA 14 Detección de Tráfico con Wireshark (Detección de Ataques DoS/DDoS)	44
FIGURA 15 Detección de Tráfico con Wireshark (Detección de Port Scanning)	45
FIGURA 16 Detección de Tráfico con Wireshark (Tráfico de DNS Sospechoso).....	46
FIGURA 17 Dataset de entrenamiento.	47
FIGURA 18 Métricas del resultado de predicciones de matriz de confusión.....	48
FIGURA 19 Reporte detallado del desempeño del modelo RF	49
FIGURA 20 <i>Métrica de predicciones</i>	50
FIGURA 21 <i>Información del Dataset</i>	51
FIGURA 22 <i>La clasificación binaria muestra resultados para ambas clases Ataque y Normal.</i>	52
FIGURA 23 <i>Matriz de confusión para datos de entrenamiento.</i>	52
FIGURA 24 <i>Comparación entre Random forest y Regresión logística.</i>	53
FIGURA 25 <i>Curvas de Precisión</i>	55
FIGURA 26 <i>Instalación del servidor y equipo en la MV</i>	77
FIGURA 27 <i>Configuración de los equipos.</i>	78
FIGURA 28 <i>Configuración de red LAN</i>	79
FIGURA 29 <i>Instalación de Isaserver</i>	80
FIGURA 30 <i>Términos y condiciones de Isa Server</i>	80
FIGURA 31 <i>Instalación completada</i>	81
FIGURA 32 <i>Consola Forefront TMG</i>	81
FIGURA 33 <i>Configuración de Directiva de Firewall</i>	82
FIGURA 34 <i>Creación de nueva regla de acceso</i>	82
FIGURA 35 <i>Agregar protocolos</i>	83
FIGURA 36 <i>Propiedades de HTTP</i>	85
FIGURA 37 <i>Agregar paginas a bloquear</i>	85
FIGURA 38 <i>Página bloqueada.</i>	86
FIGURA 39 <i>Referencia de página informativa bloqueada.</i>	88

ANEXO

ANEXO 1 Matriz de Consistencia.	68
ANEXO 2 Código fuente del desarrollo.....	70
ANEXO 3 Instalación y configuración de red para máquinas virtuales.	77

CAPITULO I: INTRODUCCIÓN

La ciberseguridad se ha transformado en una inquietud crítica para entidades y usuarios alrededor del mundo. El aumento exponencial de los datos digitales y la adopción masiva del teletrabajo tras la pandemia de COVID-19 han incrementado la necesidad de la tecnología, además de las amenazas cibernéticas. Estas amenazas, tales como el hurto de información, la modificación de datos y los accesos no permitidos, no solo amenazan la integridad de los sistemas, sino que también pueden tener consecuencias financieras y operativas graves para las empresas. Entre los ataques más comunes que enfrentan las organizaciones se encuentran el malware, que incluye variantes avanzadas como el ransomware, los ataques de día cero, que se aprovechan de fallos desconocidos en el software, y los ataques de phishing, creados para estafar a los usuarios enviando enlaces falsos y así obtener acceso a información confidencial.

A medida que las técnicas de ataque progresan en complejidad y frecuencia, los métodos de seguridad convencionales, como los sistemas de firewall y antivirus, ya no bastan para identificar intrusiones avanzadas que puedan impactar en las redes de negocios. Para proteger de manera más proactiva sus infraestructuras tecnológicas, las técnicas de machine learning ofrecen una alternativa robusta y dinámica, capaz de adaptarse y aprender de los patrones de tráfico de red para identificar comportamientos anómalos en tiempo real.

Esta investigación propone la implementación de un sistema diseñado para detectar intrusiones en redes, utilizando como bases técnicas de machine learning, con el objetivo de proteger la información corporativa. El sistema incorpora algoritmos avanzados que simplifican el análisis y la clasificación del tráfico en la red, permitiendo identificar posibles amenazas antes de que puedan ocasionar daños. Este documento detalla los objetivos, el diseño, la metodología aplicada y la respuesta obtenida en el transcurso de la implementación del sistema, con el propósito de evaluar su efectividad dentro de un entorno empresarial regulado.

CAPITULO II: PLANTEAMIENTO DEL PROBLEMA

2.1. Descripción del problema

Las empresas modernas operan en un entorno digital en el que la interconexión y la expansión de dispositivos, especialmente IoT, han amplificado la exposición a ciberamenazas. La proliferación de ataques como el robo de datos confidenciales, espionaje industrial y vulnerabilidades en infraestructuras críticas no solo compromete la integridad de los sistemas de información, sino que también genera grandes pérdidas económicas y deteriora la imagen institucional (Rodríguez et al., 2023).

Los métodos tradicionales de ciberseguridad, basados en firmas y reglas estáticas, resultan insuficientes frente a ataques emergentes y técnicas de evasión sofisticadas. En particular, estos enfoques no logran anticipar ni responder de forma eficaz a amenazas como los ataques de día cero o el malware polimórfico, que modifica su estructura para evadir la detección (Albulayhi et al., 2022). Esta situación subraya la necesidad de adoptar estrategias más dinámicas que se ajusten a la evolución constante del panorama de amenazas.

El machine learning se ha consolidado como una herramienta innovadora en el ámbito de la ciberseguridad, ya que permite analizar grandes volúmenes de tráfico de red y detectar patrones anómalos en tiempo real. Algoritmos avanzados, tales como XGBoost y los bosques aleatorios, han demostrado su capacidad para clasificar de manera precisa los comportamientos inusuales en entornos complejos y heterogéneos, ofreciendo una respuesta adaptativa frente a ataques sofisticados (Doghramachi & Ameen, 2023).

La implementación de sistemas de detección de intrusos (IDS) basados en machine learning ha facilitado también el uso de técnicas de balanceo de datos, como SMOTE, que optimizan la representación de ataques en grupos de entrenamiento y mejoran la exactitud en situaciones críticas, como las redes IoT (Qabalin et al., 2022). Esta mejora en la detección es crucial para anticipar incidentes antes de que estos puedan comprometer la infraestructura de las organizaciones.

Paralelamente, la creciente generación y transferencia de información plantea retos en términos de privacidad y cumplimiento normativo. La correcta gestión de los datos, acorde a estándares internacionales como el GDPR, es esencial para evitar sanciones legales y garantizar la confianza de los clientes. En este sentido, el machine learning no solo detecta amenazas, sino que también contribuye a diseñar estrategias de mitigación que aseguran el manejo adecuado de la información (Rodríguez et al., 2023; Tarish et al., 2024)

Por otro lado, la detección de malware incluyendo variantes como spyware, ransomware y troyanos se ha beneficiado del uso de técnicas basadas en redes neuronales y algoritmos de ensamble, los cuales ofrecen tasas de detección superiores al 98% en entornos empresariales críticos, superando las limitaciones de los métodos convencionales (Albulayhi et al., 2022; Azmee et al., 2020)

La integración de estas tecnologías en entornos IoT presenta desafíos adicionales debido a la diversidad en capacidades de hardware y la heterogeneidad de las redes. Ante este reto, han surgido modelos híbridos que combinan blockchain y machine learning, proporcionando soluciones prometedoras para garantizar tanto la integridad como la privacidad de los datos en contextos donde los estándares de seguridad varían considerablemente (Doghramachi & Ameen, 2023; Tarish et al., 2024).

Finalmente, la implementación de soluciones basadas en machine learning requiere inversiones sustanciales en infraestructura tecnológica y la capacitación especializada del personal. La continuidad en la monitorización y actualización de estos sistemas es fundamental para asegurar una respuesta ágil y eficaz ante un entorno de amenazas en constante cambio (Rodríguez et al., 2023).

2.2. Problema general

¿De qué manera un sistema de detección de intrusiones de tráfico de red basada en machine learning mejorará la protección de datos en la empresa, Lima, 2024?

2.3. Problemas específicos

- ¿Cómo influye la precisión de los algoritmos de machine learning en la detección de amenazas en tiempo real?
- ¿Cómo disminuye el sistema de detección de intrusiones basado en machine learning los falsos positivos en comparación con los sistemas convencionales?
- ¿Cómo afecta la capacidad de procesamiento del sistema la detección de amenazas en grandes volúmenes de datos?

CAPÍTULO III: DETERMINACIÓN DE OBJETIVOS

3.1. Objetivos generales

Desarrollar e implementar un sistema de detección de intrusiones en el tráfico de red basado en machine learning para garantizar la protección de los datos sensibles y la seguridad informática de la empresa, Lima, 2024.

3.2. Objetivos específicos

- Analizar y seleccionar los algoritmos de machine learning más adecuados para la detección de amenazas en entornos de red complejo.
- Diseñar y desarrollar un modelo de clasificación en tiempo real.
- Evaluar la capacidad de procesamiento y la escalabilidad del sistema.

CAPÍTULO IV: HIPOTESIS Y VARIABLES

4.1. Hipótesis generales

La implementación de un sistema de ciberseguridad basado en Machine Learning permitirá detectar en tiempo real el tráfico de red normal y malicioso, mejorando la protección de la información sensible de la empresa y reduciendo el tiempo de respuesta ante amenazas cibernéticas.

4.2. Hipótesis específicas

La implementación de técnicas de machine learning permitirá categorizar el tráfico de red con una precisión superior al 90%. Se espera que el modelo detecte de forma efectiva los patrones anómalos en el tráfico, lo que facilitará la identificación temprana de intrusiones en comparación con los métodos tradicionales.

El uso de algoritmos como Random Forest y regresión logística reducirá significativamente la tasa de falsos positivos. Esto permitirá una diferenciación más precisa entre el tráfico normal y el malicioso, optimizando la respuesta ante incidentes y mejorando la eficiencia del sistema de detección de intrusiones.

El sistema de detección de intrusiones basado en machine learning mantendrá un rendimiento óptimo y escalable bajo condiciones de procesamiento de grandes volúmenes de datos. Se espera que, incluso en entornos de alta demanda, el sistema ofrezca tiempos de respuesta adecuados sin sacrificar la precisión en la detección de amenazas.

4.3. Variables

TABLA 1

Variable dependiente e independiente.

Variable dependiente	Variable independiente
Algoritmos de machine learning.	Sistema de intrusión de trafico de red.

Nota. Elaboración propia.

4.4. Justificación

4.4.1. Teórica

La aplicación del machine learning en sistemas de ciberseguridad es fundamental ante el aumento de los ataques cibernéticos y la complejidad creciente del tráfico en las redes modernas. Como parte de la inteligencia artificial, el machine learning abre nuevas posibilidades para abordar estas amenazas de manera más eficiente (Vallejo, 2021). Por ello, se integra tecnologías innovadoras que mejoren los sistemas de seguridad, como el sistema de detección de intrusos (IDS, por sus siglas en inglés). Estos sistemas desempeñan un papel crucial en la protección de datos, ya que supervisan y analizan el tráfico en la red para identificar actividades maliciosas que puedan poner en riesgo a las organizaciones (Gutierrez-Garcia et al., 2023)

4.4.2. Práctica

En términos prácticos, muchas empresas aún no han adoptado plenamente el uso del machine learning en ciberseguridad debido a la falta de recursos necesarios para implementar tecnologías avanzadas. Estas herramientas no solo mejoran la protección de información empresarial, sino que también se adaptan y optimizan con el tiempo. La integración de un sistema de detección de intrusiones basado en machine learning incrementa a gran escala la precisión en la identificación de amenazas y reduce el tiempo de respuesta ante incidentes. De acuerdo con la investigación de (Gopalakrishnan et al., 2023), estas soluciones son altamente efectivas para clasificar actividades anómalas en redes complejas y se integran fácilmente en sistemas de seguridad preexistentes. En el ámbito regional, la implementación de este tipo de sistemas puede aportar beneficios tangibles, como la mitigación de pérdidas económicas derivadas de brechas de seguridad y el cumplimiento de normativas como la Ley de Protección de Datos Personales en Perú (Ley N.º 29733).

4.4.3. Metodológica

La presente investigación se centra en desarrollar e implementar un sistema de detección de intrusiones en el tráfico de red basado en machine learning, con especial énfasis

en la identificación de amenazas emergentes y ataques DoS/DDoS. Para lograr este objetivo, se ha adoptado un enfoque cuantitativo-experimental, que permite evaluar el rendimiento de algoritmos como Random Forest y Regresión Logística sobre el conjunto de datos CICIDS2017, garantizando la validez de los resultados obtenidos.

En cuanto a la gestión e implementación de la solución, se optó por utilizar la metodología ágil Scrum. Esta elección se fundamenta en las siguientes razones:

- **Adaptabilidad y Flexibilidad:** En el entorno de ciberseguridad, donde las amenazas evolucionan rápidamente y los requisitos pueden cambiar en función de los hallazgos en tiempo real, las metodologías tradicionales como el modelo en cascada resultan demasiado rígidas. Scrum, por su parte, permite iteraciones cortas en sprints y la revisión continua del producto, lo cual es crucial para ajustar y optimizar el sistema de detección a medida que se identifican nuevos patrones de tráfico y amenazas emergentes.
- **Iteración y Retroalimentación:** La implementación de sistemas basados en machine learning requiere constantes ajustes en los parámetros de los modelos y en la integración de nuevos datos. Scrum facilita la incorporación de retroalimentación a lo largo de todo el proceso mediante reuniones diarias, revisiones de sprint y retrospectivas, asegurando que cada iteración del sistema se alinee con los objetivos de protección de datos y se optimice en función de las métricas obtenidas en las fases de prueba.
- **Respuesta Ágil ante Incertidumbres:** Dado el dinamismo del tráfico de red y la variedad de ataques cibernéticos (incluyendo ataques DoS/DDoS), es imprescindible contar con un marco que permita responder ágilmente a imprevistos. La capacidad de Scrum para gestionar el cambio y priorizar funcionalidades críticas a corto plazo resulta especialmente adecuada para este estudio, ya que se requieren iteraciones rápidas para evaluar la efectividad del sistema y realizar los ajustes necesarios.

Si bien existen otros marcos metodológicos ampliamente utilizados en proyectos de machine learning y análisis de datos, como CRISP-DM, la elección de Scrum se justifica en este caso por su capacidad para gestionar de forma iterativa la implementación del sistema y adaptarse a las condiciones cambiantes del entorno de ciberseguridad. Esta integración de Scrum con el enfoque cuantitativo-experimental asegura que tanto el análisis de datos como

la fase de implementación se realicen con la agilidad y rigor necesarios para enfrentar las complejidades del tráfico de red y las amenazas emergentes.

CAPÍTULO V: MARCO TEORICO Y ANTECEDENTES

5.1. Marco Teórico

5.1.1. Protección de Datos en Empresas

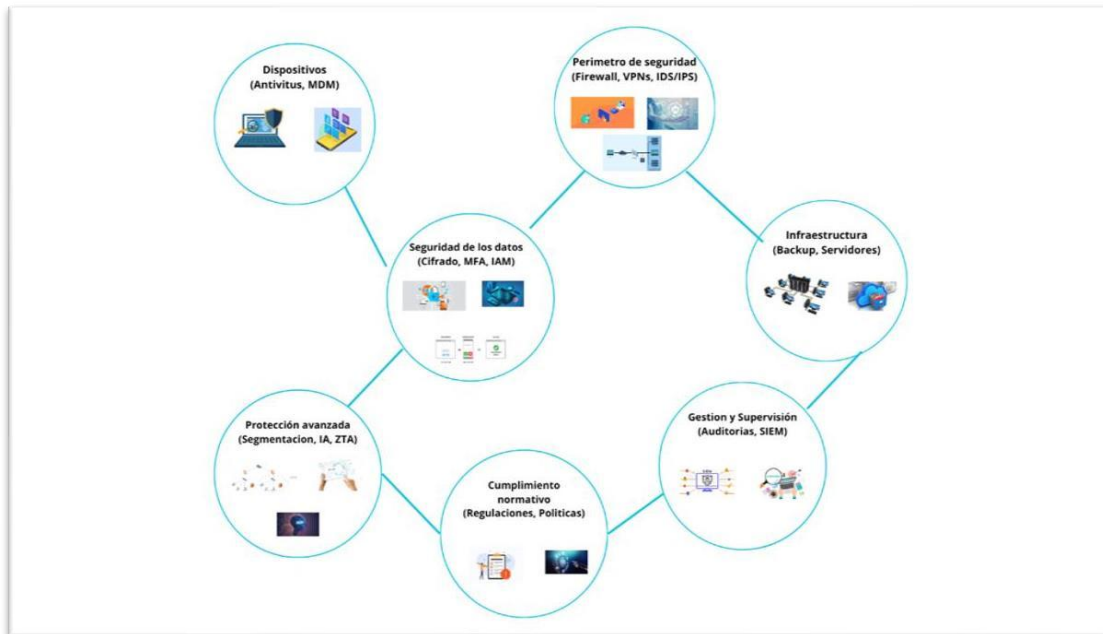
La gestión de la protección de datos en las organizaciones privadas cubre un conjunto de políticas diseñadas para garantizar la seguridad, privacidad e integridad de los datos personales, financieros y comerciales dentro de la entidad. En la era digital, esta protección ha adquirido una relevancia crucial debido al incremento de ataques cibernéticos y al papel fundamental de los datos como un activo corporativo, que no solo permite el cumplimiento normativo, sino que también previene posibles sanciones (Fernández Acevedo, 2024).

Como se ilustra en la Figura 1, la arquitectura para proteger la información empresarial clasifica sus nodos de la siguiente manera.

- **Dispositivos:** Este aspecto se enfoca en proteger los dispositivos finales, tales como computadoras, laptops y teléfonos móviles, mediante el uso de antivirus o mediante políticas de gestión que implemente la empresa.
- **Seguridad de los datos:** Este componente se encarga de respaldar la privacidad y protección de los datos mediante la gestión de accesos, la implementación de autenticación multifactorial y el uso de tecnologías de cifrado.
- **Perímetro de seguridad:** Regula el tráfico de datos en la red mediante herramientas como cortafuegos, redes privadas virtuales (VPN) y sistemas diseñados para la detección de intrusos.
- **Protección avanzada:** Con el fin de proteger la información y bloquear posibles amenazas, se recurre a innovadoras tecnologías como la IA (inteligencia artificial) y a la implementación de arquitecturas como Zero Trust.
- **Infraestructura:** Asegura la disponibilidad de la información a través de servidores debidamente protegidos.
- **Gestión y Supervisión:** Esta área facilita la monitorización de los eventos de seguridad en tiempo real mediante auditorías y aplicaciones especializadas como los sistemas SIEM (Security Information and Event Management).

FIGURA 1

Arquitectura de protección de la información empresarial.



Nota. Elaboración propia.

5.1.2. Seguridad de la Información

La privacidad de información juega un rol fundamental en la a la hora de salvaguardar los datos personales de las empresas. Con el avance tecnológico hoy en día, las amenazas cibernéticas han evolucionado, por ello es necesario implementar sistemas de protección cada vez más sofisticados para detectar y mitigar los ataques de manera eficiente. De acuerdo con la norma ISO/IEC, la seguridad de la información se refiere a una serie de procedimientos, buenas prácticas y métodos diseñados para la protección de la información y los sistemas de información, protegiéndolos de accesos no permitidos, uso inadecuado, divulgación o interrupción que puedan poner en riesgo su integridad (Vega Briceño, 2021).

5.1.3. Pilares de la Seguridad de la Información

Como se muestra en la Figura 2, la confidencialidad asegura que solo las personas autorizadas puedan tener acceso a la información. La integridad se encarga de proteger la precisión y la totalidad de los datos, así como los procesos que los manejan, y la disponibilidad

garantiza que los usuarios autorizados puedan acceder a dicha información cuando la necesiten.

FIGURA 2

Pilares de seguridad de información.



Nota. Adaptado de Seguridad de la información, (Vega Briceño, 2021, p. 12,13).

5.1.4. Amenazas comunes

Según Angeles Caballero et al. (2024), las amenazas son solo algunas de las maneras más habituales en las que la protección de la información puede ser vulnerada.

Una amenaza se refiere a cualquier circunstancia, elemento o escenario que simboliza un riesgo potencial o amenaza para un individuo, organización o sistema. Estos pueden presentarse de diversas maneras, tales como amenazas económicas y tecnológicas. Es importante identificar y entender las amenazas para elaborar estrategias de prevención y protección eficaces, entre ellas tenemos los siguientes:

- **Malware:** Dentro de los tipos más habituales de malware se encuentran los virus, que se difunden entre archivos, el ransomware, que cifra los datos del usuario y solicita un rescate para su liberación y el spyware, que recolecta datos delicados sin el permiso del usuario (Vega Briceño, 2021).

- **Phishing:** Este método de ciberataque se aplica para confundir a las personas y conseguir acceso no permitido a datos sensibles, tales como contraseñas, números de tarjetas bancarias o credenciales de acceso. Estos ataques se llevan a cabo por medio de emails, mensajes de texto o páginas web engañosas que simulan fuentes fiables, con el objetivo de que el usuario divulgue su información personal (Guana-Moya et al., 2022).
- **Ataques de Denegación de Servicio (DoS/DDoS):** Los ataques de Denegación de Servicio (DoS) buscan abrumar un sistema o red con tráfico excesivo, lo que impide que los usuarios legítimos puedan acceder a ellos. En los ataques Distribuidos de Denegación de Servicio (DDoS), los ataques provienen de múltiples fuentes, lo que afecta de manera significativa la disponibilidad de servicios o sitios web, generando pérdidas tanto financieras como de reputación (Manotas Pacheco, 2024).
- **Amenazas Internas:** Este tipo de amenaza proviene de personas dentro de la organización, como empleados, empleadores o colaboradores, quienes, ya sea de forma intencionada, por desconocimiento o por simple negligencia, pueden exponer o comprometer información sensible. Las amenazas internas incluyen desde el acceso no autorizado por parte de un empleado con privilegios elevados hasta la divulgación accidental de información debido a una capacitación inadecuada o el incumplimiento de políticas de seguridad (Puentes Becerra, 2020).

5.1.5. Sistema de Gestión de Seguridad de la Información (SGSI)

Un Sistema de Gestión de Seguridad de la Información (SGSI) es una estrategia clave que permite gestionar de manera eficiente la protección de los datos, garantizando su privacidad, integridad y disponibilidad. Basado en estándares como la ISO/IEC 27001, el SGSI incluye políticas, procedimientos y controles que siguen el ciclo de mejora continua: Planificar, Hacer, Verificar y Actuar. Su principal propósito es identificar, reducir y monitorear los riesgos de seguridad, al mismo tiempo que fortalece la confianza de los clientes y socios (Sánchez López, 2024).

FIGURA 3
Ciclo de mejora continua (PDCA).



Nota. Adaptada de Metodología PDCA utilizada para la elaboración de un manual de procesos en el área de kit's en un almacén de herramientas (Sánchez López, 2024, p. 4).

En la Figura 3 el ciclo PDCA se emplea para aplicar modificaciones de forma organizada y eficaz. Se detallan a continuación las cuatro fases fundamentales:

- **Planificar (Plan):** En esta fase se detectan dificultades, posibilidades de mejora o zonas críticas.
- **Hacer (DO):** En esta etapa se lleva a cabo las acciones programadas, pero usualmente a escala reducida como proyectos piloto o pruebas.
- **Verificar (Check):** Esta etapa mide la efectividad de las medidas aplicadas.
- **Actuar (Act):** En esta última etapa se toman decisiones fundamentadas en los hallazgos de la verificación.

5.1.6. Tráfico de red

El tráfico de red hace referencia al flujo de información que circula a través de una red de computadoras, representado por paquetes de datos que se envían y reciben entre dispositivos conectados. Este intercambio abarca diferentes formas de comunicación, como navegar por internet, compartir archivos, enviar correos electrónicos y realizar videoconferencias, entre otros. Para asegurar un rendimiento óptimo, es crucial que la red

mantenga un ancho de banda adecuado, con baja latencia, y ofrezca un servicio de alta calidad (León et al., 2022).

Clasificación del tráfico de red

La clasificación facilita la implementación de políticas de calidad de servicio (QoS), que priorizan ciertos tipos de tráfico, optimizan el uso del ancho de banda y mejoran la experiencia del usuario. Su objetivo es identificar y mitigar acciones perjudiciales, asegurando la protección y estabilidad de la red (León et al., 2022).

Existen cuatro métodos de clasificación de tráfico:

Clasificación de tráfico basado en puertos: Este método detecta el tráfico examinando los números de puertos empleados en los títulos o encabezados de los paquetes. Cada protocolo estándar cuenta con puertos determinados (como HTTP utiliza el puerto 80 y HTTPS el 443).

Clasificación de tráfico basado en firmas o patrones (Deep Packet Inspection DPI): Este método analiza el contenido de los paquetes, buscando identificar patrones particulares en los datos para identificar el protocolo o la aplicación.

Clasificación de tráfico basado en estadísticas: Este método se relaciona con los atributos del tráfico, tales como la dimensión de los paquetes, la frecuencia, la dirección o los periodos entre las transmisiones.

Clasificación de tráfico basado en el comportamiento: Este método analiza patrones de uso y comportamiento de la red, detectando anomalías o actividades particulares que están vinculados a determinados tipos de tráfico.

A continuación, se muestra en la Tabla 2 las ventajas y desventajas de cada clasificación del tráfico de red.

TABLA 2*Ventajas y Desventajas de la Clasificación del tráfico de red.*

Clasificación	Ventajas	Desventajas
Basado en puertos	Es rápido y sencillo de implementar.	No es confiable si los puertos son modificados o si el tráfico está cifrado.
Basado en firmas o patrones	Alta precisión para identificar aplicaciones.	Consume más recursos, resulta intrusivo y no opera con tráfico encriptado.
Basado en estadísticas	Es beneficioso incluso cuando el tráfico se encuentra encriptado.	Puede generar falsos positivos por lo que se requiere un entrenamiento previo.
Basado en el comportamiento	Efectivo para identificar tráfico malicioso o desconocido.	Puede requerir tiempo para establecer un modelo base y generar falsos positivos.

Nota. Elaboración propia.

5.1.7. Sistema de Detección de Intrusiones (IDS)

Un Sistema de Detección de Intrusiones (IDS) es una herramienta de ciberseguridad diseñada para monitorear y analizar el tráfico de la red o las actividades de un sistema, con el propósito de identificar comportamientos sospechosos o maliciosos. Este sistema funciona como un vigilante que busca patrones que puedan indicar intentos de infiltración, acceso no autorizado o amenazas a la integridad, privacidad y disponibilidad de la información (Caizapanta González, 2022).

Clasificación de IDS

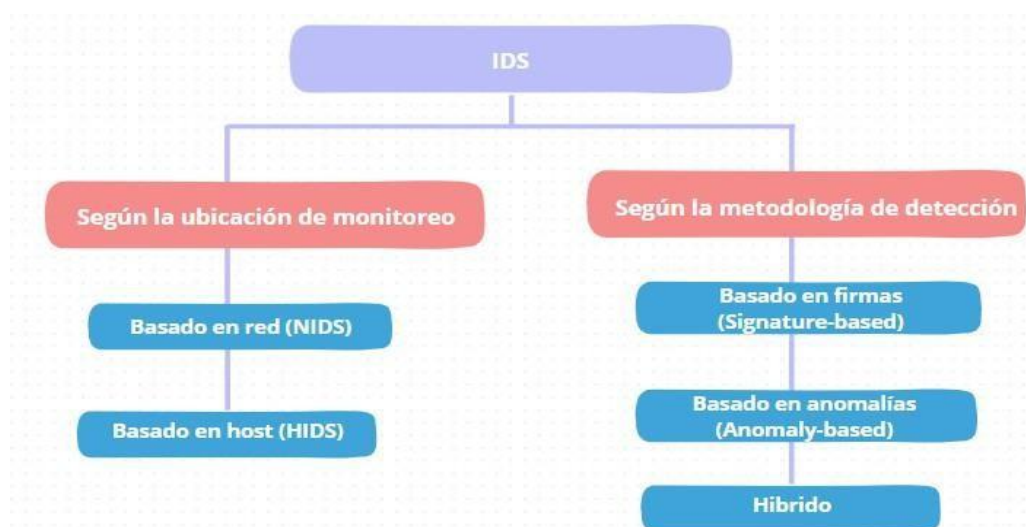
Según el Gobierno de España (2017), existen diversos tipos de IDS, los cuales se clasifican de acuerdo con el lugar donde se realiza la vigilancia: aquellos basados en red (NIDS), que examinan el tráfico en busca de acciones sospechosas, y los basados en host

(HIDS), que monitorean eventos y archivos en sistemas individuales. Además, según su método de detección, pueden fundamentarse en firmas, al contrastar ataques con patrones establecidos previamente; en anomalías, al identificar acciones atípicas en contraposición al comportamiento habitual; o en enfoques híbridos, que combinan ambas tácticas.

- **Basados en firmas (Signature-based):** Identifican intrusiones al comparar el tráfico o los sucesos con una base de datos de patrones de ataques de firmas. Son eficaces frente a amenazas ya existentes, pero no detectan ataques emergentes.
- **Basados en anomalías (Anomaly-based):** Monitorean el flujo de tráfico para detectar anomalías de un comportamiento normal previamente establecido. Son eficaces para identificar amenazas desconocidas, aunque pueden producir falsos positivos.
- **IDS Basado en Red (NIDS):** Administra el flujo de datos en una red para detectar acciones sospechosas. Se ponen en lugares clave, como switches o routers, para examinar paquetes en tiempo real.
- **IDS Basado en Host (HIDS):** Monitorean las operaciones en un dispositivo o servidor concreto, tales como cambios en archivos, registros o intentos de acceso no permitido.

A continuación, en la Figura 4 se visualizan según su clasificación.

FIGURA 4
Clasificación de IDS.



Nota. Elaboración propia.

5.1.8. Machine Learning en Ciberseguridad

El machine learning ha adquirido una gran relevancia en la ciberseguridad debido a su capacidad para analizar grandes cantidades de datos e identificar patrones inusuales que podrían indicar amenazas o ataques. Las técnicas de machine learning permiten crear sistemas de detección de intrusiones (IDS) más precisos y efectivos, capaces de identificar amenazas nuevas y avanzadas, como ataques de día cero o malware desconocido. Entre los métodos de machine learning utilizados en ciberseguridad, se destacan el aprendizaje supervisado, que se usa para clasificar patrones conocidos como amenazas, y el aprendizaje no supervisado, que identifica comportamientos anómalos sin la necesidad de información previamente etiquetada. Estos enfoques se aplican en tareas clave, como la detección de fraudes, el análisis de comportamientos y la autenticación de usuarios en tiempo real (Alvarado et al., 2022)

TABLA 3

Diferentes tipos de machine learning que se aplican en ciberseguridad.

Aspecto	Aprendizaje supervisado	Aprendizaje no supervisado	Aprendizaje por refuerzo
Definición	El modelo aprende de datos etiquetados para clasificar o predecir datos similares.	El modelo identifica patrones en datos no etiquetados para encontrar anomalías o comportamientos atípicos.	El modelo aprende mediante pruebas y recompensas, mejorando su respuesta a través de ensayo y error.
Ventajas	Alta precisión en detección de amenazas conocidas. Proceso de entrenamiento más rápido con datos etiquetados.	Detecta amenazas desconocidas o de día cero. No requiere datos etiquetados, adaptándose a cambios rápidos.	Puede optimizar decisiones en tiempo real. Se adapta continuamente a entornos dinámicos.
Desventajas	Requiere gran cantidad de datos	Puede tener una mayor tasa de	Entrenamiento más complejo y costoso.

	etiquetados. No detecta eficazmente amenazas nuevas o no etiquetadas.	falsos positivos. Más difícil de entrenar y evaluar con precisión.	La recompensa y castigo deben estar claramente definidos.
Aplicaciones en ciberseguridad	Detección de malware. Filtrado de spam. Clasificación de intrusiones.	Detección de anomalías en redes. Identificación de ataques de día cero. Monitorización de comportamientos.	Respuesta automatizada a incidentes. Protección de puntos finales en tiempo real. Seguridad en IoT.

Nota: Elaboración propia.

Arquitectura de un Sistema de Intrusión Basado en Machine Learning

Según Saba et al. (2022), un sistema de detección de intrusiones de red basado en machine learning se compone de los siguientes componentes:

- **Recolección de Información:** Consiste en recopilar en tiempo real los paquetes de red para su posterior análisis, incluyendo detalles sobre la procedencia, el destino y el tamaño de los paquetes.
- **Preprocesado de Información:** Implica la depuración y normalización de los datos para mejorar la calidad y precisión del modelo, lo que puede incluir la eliminación de información redundante y la modificación de ciertos atributos.
- **Formación del Modelo:** Se implementan algoritmos de machine learning, como redes neuronales, árboles de decisión y máquinas de soporte vectorial, para crear un modelo que sea capaz de identificar intrusiones.
- **Identificación y Clasificación:** Se realiza un análisis en tiempo real del tráfico de red, clasificando los patrones para detectar posibles amenazas.

- **Alerta y Reacción:** Se generan alertas y se implementan respuestas automáticas frente a las amenazas detectadas, lo que permite tomar medidas antes de que se produzca un daño significativo.

Algoritmos de Machine Learning utilizados en IDS

Según Valdezate Alvarez (2020), los algoritmos de machine learning desempeñan un papel crucial en la identificación de patrones complejos y en la adaptación a nuevas amenazas. En consecuencia, se destacan algoritmos como los árboles de decisión, que permiten clasificar la información de manera jerárquica; Random Forest, el cual, mediante el uso de técnicas de ensamble, mejora la robustez y la precisión en la detección; y los Support Vector Machines (SVM), reconocidos por maximizar el margen entre clases y ofrecer una separación más clara de los patrones. Además, otros enfoques, como las redes neuronales y el algoritmo k-nearest neighbors (KNN), contribuyen significativamente a mejorar la efectividad en la detección de intrusiones, ya que facilitan la identificación de comportamientos anómalos en el tráfico de red, permitiendo un análisis más detallado y adaptativo ante la evolución de las amenazas.

- **Redes Neuronales Artificiales (ANN):** Útiles para clasificar tráfico de red con alta precisión.
- **Árboles de Decisión:** Para clasificación de tráfico.
- **Random Forest:** Proveen interpretabilidad y rapidez en la clasificación de datos de tráfico.
- **Máquinas de Soporte Vectorial (SVM):** Eficaces en la separación de patrones maliciosos y normales.
- **Algoritmos de Clustering:** Como K-means y DBSCAN, útiles en aprendizaje no supervisado para detectar patrones de anomalía.
- **Deep Learning:** Para análisis de comportamiento y detección de anomalías.

5.1.9. Dataset

Un dataset (o conjunto de datos) es una colección organizada de información que se utiliza para análisis, modelado o el entrenamiento de algoritmos. Estos conjuntos de datos pueden incluir diferentes tipos de información, como números, texto, imágenes o sonidos, y

generalmente están estructurados en tablas, matrices u otras formas similares. En un dataset, cada fila representa una observación o instancia, mientras que cada columna corresponde a un atributo o característica de esa observación (Menéndez Domínguez et al., 2022).

Características de un dataset

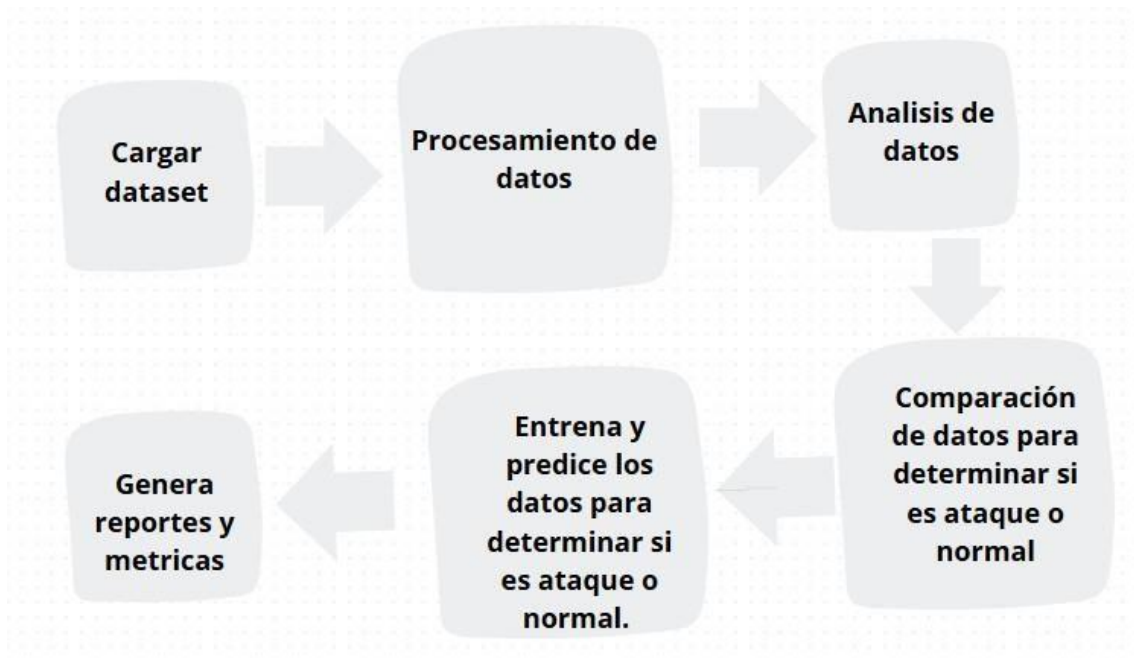
Según Menéndez Domínguez et al. (2022), mencionan que las características importantes de un dataset incluyen su tamaño, su formato, su calidad y la forma en que se ha recolectado, las cuales son las siguientes:

- **Estructura:** Puede ser tabular como en una hoja de cálculo o no estructurado como imágenes o textos.
- **Formato:** Almacenados en formatos como CSV, Excel, JSON, SQL, entre otros.
- **Etiquetado:** Algunos datasets incluyen etiquetas supervisados, mientras que otros no supervisados.
- **Tamaño:** El tamaño puede variar desde pequeños conjuntos para análisis simples hasta grandes volúmenes usados en big data.

CICIDS2017 (Instituto Canadiense de Ciberseguridad para la Detección de Intrusiones 2017) es un conjunto de datos ampliamente reconocido y utilizado en el ámbito de la investigación y desarrollo de sistemas de detección de intrusiones (IDS). Este conjunto cubre una amplia gama de escenarios de tráfico en la red, tanto normales como maliciosos. El dataset CICIDS2017 está diseñado para apoyar el progreso de técnicas avanzadas, como el machine learning y Deep Learning, y facilita la comparación de algoritmos y modelos en términos de precisión, eficacia y capacidad para detectar cualquier incidencia en la red (Montes Gil et al., 2023).

FIGURA 5

Arquitectura de análisis de datos.



Nota. Elaboración propia.

Como se muestra en la Figura 5, la arquitectura del IDS describe el proceso de ejecución de un dataset, que incluye la captura de la información necesaria para su posterior análisis. Este proceso se utiliza para entrenar el modelo y generar los resultados en forma de reportes o métricas, permitiendo evaluar el desempeño y la efectividad del sistema de detección de intrusiones.

5.1.10. Vulnerabilidades

Las vulnerabilidades son fragilidades o errores en sistemas, programas informáticos, redes o procedimientos de seguridad que pueden ser aprovechadas por intrusos para conseguir acceso no permitido, manejar datos o provocar perjuicios. Estas deficiencias pueden derivarse de fallos en el código, ajustes equivocados, o fallos de seguridad en el diseño de sistemas (Bishop et al., 2019).

5.2. Estado del Arte

Según Al Hasan y Kadhum Hamza (2023), el aumento de los ataques cibernéticos y la diversificación de las amenazas digitales han impulsado el desarrollo de sistemas de

ciberseguridad avanzados; en este sentido, los IDS basados en machine learning se han mostrado altamente efectivos y adaptables para la protección de redes en entornos empresariales, ya que identifican patrones y comportamientos anómalos en el tráfico de datos, lo que permite prevenir intrusiones y optimizar la seguridad de la infraestructura tecnológica.

De igual manera, Al Hasan y Kadhum Hamza (2023) exploraron el desarrollo de un IDS mejorado para redes de sensores inalámbricos (WSN) con el objetivo de aumentar la precisión en la detección y reducir las falsas alarmas; para ello, aplicaron técnicas de reducción de dimensionalidad, como la descomposición en valores singulares (SVD) y el análisis de componentes principales (PCA), utilizando los datasets WSN-DS y UNSW-NB15, lo que permitió obtener una precisión superior al 99% en la detección de ataques comunes en WSN (por ejemplo, blackhole y flooding).

Asimismo, Ayuba et al. (2024) propusieron un modelo que combinó PCA con algoritmos de machine learning, tales como AdaBoost y LogitBoost, e integraron metodologías de ensamble como Random Forest y Bagging; de esta forma, al analizar datasets como WSN-DS, NSL-KDD y UNSW-NB15, se evidenció que la integración de PCA con Random Forest alcanzó una precisión del 100%, minimizando la varianza y evitando el sobreajuste.

Por otro lado, Sajid Farooq et al. (2023) desarrollaron el enfoque IDS-FMLT, destinado a detectar intrusiones en redes heterogéneas y proteger contra ataques tanto conocidos como desconocidos; en este estudio se aplicaron algoritmos de machine learning junto con validación cruzada en un entorno experimental, alcanzando una precisión del 95.18% en la detección en tiempo real.

Además, Chimphee y Chimphee (2023) exploraron el uso del machine learning para mejorar la detección de intrusiones basadas en anomalías en redes con grandes volúmenes de datos; utilizando el Perceptrón Multicapa (MLP) y Random Forest sobre el dataset CSE-CIC-IDS-2018, demostraron que el preprocesamiento y la selección de características optimizan la detección, siendo crucial la preselección de atributos para manejar datos grandes y diversos.

De igual forma, Baklizi et al. (2024) evaluaron la eficacia de algoritmos como Random Forest, k-nearest neighbors y Naive Bayes en aplicaciones web; utilizando el dataset CIC-IDS2017, se observó que Random Forest y k-nearest neighbors presentaron niveles de precisión muy altos de 99.49% y una tasa de recuperación del 100%, respectivamente, lo que respalda su idoneidad para detectar intrusiones en tráfico web en entornos complejos.

Por otro lado, Lezcano Gil et al. (2023) realizaron una revisión sistemática que destacó la importancia de medidas como el cifrado de datos y la autenticación multifactorial en la protección de la información en la nube, evidenciando que estos controles son esenciales para prevenir violaciones de seguridad y proteger la privacidad del usuario.

De igual modo, Huaraca Nuñez et al. (2024) llevaron a cabo una revisión sistemática de estudios publicados entre 2018 y 2024 sobre técnicas de machine learning aplicadas a IDS, subrayando el creciente uso de estas técnicas y su capacidad para adaptarse a ataques emergentes, especialmente cuando se combinan con procesos de selección de características.

Asimismo, Suárez y Rodríguez (2023) analizaron la efectividad de los IDS basados en machine learning en la reducción de falsos positivos y en la provisión de protección en tiempo real en entornos cibernéticos de rápida evolución, enfatizando la necesidad de una gestión continua para mantener actualizadas las capacidades de estos sistemas.

Por otro lado, Luna García (2023) examinó la implementación de la inteligencia artificial en la modernización del Estado peruano, observando que las principales aplicaciones de la IA se centran en mejorar la administración pública y la seguridad de la información, a pesar de enfrentar desafíos técnicos e infraestructurales.

Finalmente, Mostacero Gamboa (2020) realizó una revisión sistemática sobre los controles y mecanismos en la gestión de la seguridad de red basada en IDS, identificando que la integración de técnicas de machine learning resulta esencial para mejorar la efectividad de los IDS frente a amenazas en constante evolución, lo que subraya la importancia de un monitoreo continuo.

En conjunto, la literatura indica que la incorporación de técnicas avanzadas de machine learning, junto con métodos de reducción de dimensionalidad y selección de características, contribuye de manera significativa a mejorar la detección de intrusiones y a optimizar la seguridad de las redes en entornos empresariales altamente vulnerables.

CAPITULO VI: METODOLOGÍA DE LA INVESTIGACIÓN

6.1. Diseño de la investigación

El diseño de la investigación fue cuantitativo-experimental, ya que se emplearon algoritmos de machine learning para medir su eficacia en la detección de tráfico anómalo y la protección de datos confidenciales en redes empresariales. Según (Ruiz Huaraz et al., 2023), el diseño cuantitativo experimental permite manipular de manera controlada las variables independientes (en este caso, los algoritmos y sus hiperparámetros) para observar su efecto sobre la variable dependiente (capacidad de detección de intrusiones).

Dentro del ámbito de la ciberseguridad, este enfoque es especialmente útil para probar diferentes configuraciones de modelos y determinar cuál ofrece mejores resultados en escenarios de ataque simulados.

6.2. Tipo de la investigación

El tipo de investigación fue aplicada, ya que se busca desarrollar un sistema práctico y funcional que permita resolver un problema específico: la detección de amenazas cibernéticas en redes y la protección de la información comercial mediante machine learning. Este enfoque tiene como objetivo la creación de una solución que no solo sea teórica, sino que tenga una aplicación directa y concreta en el mundo real.

De acuerdo con (Ruiz Huaraz et al., 2023) la investigación aplicada se orienta hacia la resolución de problemas concretos que afectan a la sociedad o al entorno empresarial. A diferencia de la investigación básica, que se centra en la generación de conocimiento sin una aplicación inmediata, la investigación aplicada busca ofrecer soluciones prácticas que puedan ser implementadas para mejorar situaciones específicas. En este caso, el propósito es desarrollar un sistema eficiente y viable que no solo identifique y proteja las redes contra posibles amenazas, sino que también sea una herramienta útil para las empresas que buscan mejorar su seguridad informática.

6.3. Alcance

El estudio posee un alcance longitudinal, ya que se investiga cómo el sistema se adapta a cambios en el tráfico de red, amenazas cibernéticas emergentes y actualizaciones

en los algoritmos de machine learning a lo largo del tiempo (Ruiz Huaraz et al., 2023). Este enfoque permite analizar la evolución de los resultados y la capacidad del modelo para mantenerse eficaz ante escenarios cambiantes.

6.4. Enfoque

El enfoque fue cuantitativo, dado que se recurre a la recolección y el análisis de datos numéricos (por ejemplo, tasa de falsos positivos, precisión, exactitud). Según Hadi et al. (2023), este método se basa en la objetividad y la replicabilidad, utilizando herramientas estadísticas y experimentales para validar las hipótesis planteadas.

6.5. Población y muestra

- **Población:** Todos los eventos y patrones de tráfico de red que ocurren en la empresa.
- **Muestra:** Se seleccionaron muestras de 30 días hábiles de eventos de tráfico de red, previamente clasificados como normales o anómalos, para entrenar y evaluar los algoritmos de machine learning.

Para que los resultados obtenidos puedan extrapolarse, se emplearon métodos de muestreo que aseguran la representatividad de los datos (Hadi et al., 2023).

6.6. Instrumento de Medición y Técnica

Se utilizó la técnica de observación para recolectar datos en tiempo real del tráfico de red, complementada con el instrumento de checklist (lista de verificación), donde se registraron condiciones y actividades específicas durante la fase de implementación y pruebas. De esta forma, se garantizó la coherencia y trazabilidad de los datos analizados (Medina et al., 2023).

6.7. Recolección del Conjunto de Datos y Variables Utilizadas

Para este estudio, se empleó el conjunto de datos etiquetado CICIDS2017, que incluye tanto tráfico normal como diversos tipos de ataques (DoS, DDoS, brute force, infiltración, etc.). La columna "Label" distingue entre tráfico benigno y malicioso, lo que facilita la clasificación.

Estructura y Etiquetado

- Cada registro representa un flujo de red con atributos como dirección IP de origen, IP de destino, puertos, duración, bytes transmitidos y otros indicadores relevantes.
- Se agruparon las etiquetas en dos categorías principales: normal y malicioso, manteniendo el detalle de cada tipo de ataque para análisis específicos.

Distribución de Clases

- El dataset presenta un desequilibrio significativo en la proporción de tráfico normal versus tráfico malicioso, especialmente para algunos tipos de ataque de baja frecuencia.
- Para evitar que el modelo aprenda patrones triviales y descuide las clases minoritarias, se aplicaron técnicas de balanceo, como SMOTE (Synthetic Minority Oversampling Technique), en aquellas clases donde la diferencia era más pronunciada.

Preprocesamiento

- Se eliminaron registros duplicados e inconsistentes.
- Se normalizaron variables continuas como Flow Duration y Flow Bytes/s.
- Se seleccionaron las características con mayor relevancia, según estudios previos y la literatura del Estado del Arte, descartando aquellas que no aportaban información significativa o que inducían ruido.

6.8. Proceso Experimental de Entrenamiento y Prueba de los Modelos

División de Datos (Train/Test)

Para evitar el sobreajuste y evaluar de forma objetiva la capacidad de generalización de los modelos, se dividió el conjunto de datos en dos partes: 80% para entrenamiento y 20% para prueba. Esta separación se realizó asegurando la presencia de la clase minoritaria en ambos subconjuntos, lo cual garantiza que el modelo enfrente muestras representativas de todos los tipos de tráfico, incluidos los menos frecuentes, en cada fase del proceso.

Entrenamiento

En la etapa de entrenamiento, se emplearon algoritmos de machine learning como Random Forest y Regresión Logística, cuyos hiperparámetros se ajustaron mediante validación cruzada (k-fold). Este enfoque permitió utilizar diferentes particiones del conjunto de entrenamiento para calibrar y validar los modelos, incrementando la fiabilidad de las estimaciones de rendimiento antes de su evaluación final.

Evaluación con Datos No Vistos

Una vez concluidos los ajustes de hiperparámetros, se evaluaron los modelos en el conjunto de test, es decir, datos no empleados en la fase de entrenamiento. Durante esta evaluación se calcularon métricas clave como precisión (Precisión), tasa de falsos positivos (FPR), tasa de falsos negativos (FNR) y exactitud (Accuracy), lo que facilitó la cuantificación del desempeño real de los algoritmos ante situaciones desconocidas.

Control del Desbalance

Además de la aplicación de SMOTE en la fase de preprocesamiento, se monitorearon métricas por clase (como sensibilidad y especificidad) para comprobar que los modelos no descuidaran las categorías minoritarias. Este control de equilibrio resultó esencial para asegurar que el sistema ofreciera un desempeño estable, detectando intrusiones de baja frecuencia sin generar un exceso de falsos positivos.

Documentación y Replicabilidad

Finalmente, se registraron los detalles de cada transformación de datos, los valores de hiperparámetros seleccionados y las versiones de las librerías de Python utilizadas. Con ello, se facilita la reproducibilidad de la investigación, de modo que otros interesados puedan

replicar el experimento o comparar resultados bajo condiciones similares, reforzando la transparencia y la validez del estudio.

6.9. Métricas de Selección

Para evaluar el rendimiento del sistema de intrusión de tráfico de red basado en machine learning, se emplearon indicadores clave:

Tasa de detección (Detection Rate - DR): Porcentaje de ataques identificados exitosamente en comparación con el total de ataques ejecutados.

Formula:

$$DR = \frac{\text{Ataques detectados}}{\text{Total de ataques}} * 100$$

Tasa de falsos positivos (False Positive Rate - FPR): Proporción de eventos normales que son incorrectamente clasificados como ataques.

Fórmula:

$$FPR = \frac{\text{Eventos normales como ataques}}{\text{Total de eventos normales}} * 100$$

Tasa de falsos negativos (False Negative Rate - FNR): Porcentaje de ataques no detectados.

Fórmula:

$$FNR = \frac{\text{Ataques no detectados}}{\text{Total de ataques}} * 100$$

Precisión (Precisión): Medida de qué tan relevantes son las alertas generadas.

Fórmula:

$$P = \frac{\text{Ataques detectados}}{\text{Total de ataques generados}} * 100$$

Exactitud (Accuracy): Proporción de clasificaciones correctas, tanto normales como maliciosas.

Fórmula:

$$E = \frac{\text{Eventos correctamente clasificados}}{\text{Total de eventos analizados}} * 100$$

Estos indicadores permiten analizar de forma integral la eficacia del sistema para detectar amenazas y la probabilidad de incurrir en errores, aspectos fundamentales para la ciberseguridad.

6.10. Metodología de medición

La presente metodología de medición se centra en tres aspectos esenciales para evaluar el rendimiento de un sistema de intrusión de tráfico de red basado en machine learning: la identificación precisa de intrusiones, la exactitud de los modelos de Machine learning y la capacidad de protección de la información corporativa. En este contexto, la validez de juicio de expertos resulta fundamental, pues implica la participación de profesionales con experiencia en seguridad informática y machine learning, quienes validan los enfoques técnicos y metodológicos, garantizando la pertinencia de los algoritmos seleccionados y la efectividad de las soluciones propuestas (Almada Martínez, 2019).

Asimismo, la validez de contenido asegura que el instrumento de medición abarque todos los aspectos relevantes para la detección de intrusiones y la protección de datos, verificando que los ítems o preguntas reflejen adecuadamente las dimensiones críticas del fenómeno estudiado. De esta manera, se confirma que el sistema de intrusión de tráfico de red incluya todas las métricas necesarias para un análisis integral del desempeño y la mitigación de amenazas.

Por último, la confiabilidad del instrumento se refiere a la consistencia y estabilidad de los resultados en diferentes contextos y momentos (Borjas García, 2020). Para evaluarla, se pueden aplicar métodos como el test-retest, la consistencia interna o la validación cruzada (Cross-Validation), asegurando que las mediciones no presenten variaciones indeseadas y

que el sistema mantenga un rendimiento predecible ante diversos escenarios de tráfico y volúmenes de datos.

6.11. Selección de la metodología

La implementación de un sistema de intrusión de tráfico de red basado en machine learning requiere seleccionar una metodología adecuada que se alinee con los objetivos del proyecto, con el fin de garantizar su éxito y eficiencia. En este caso, se optó por la metodología ágil SCRUM, que se caracteriza por su capacidad para ajustarse a los objetivos del proyecto en cada etapa, permitiendo iteraciones rápidas y adaptaciones sobre la marcha.

El enfoque ágil es una metodología de gestión de proyectos y desarrollo de software que destaca la importancia de la flexibilidad, la colaboración, la entrega constante de resultados y la rápida adaptación a cambios. Esta metodología promueve la creación continua de valor, la mejora constante y la cooperación dentro del equipo, lo cual es esencial en un entorno dinámico y de alta incertidumbre como el de la ciberseguridad (Flores-Cerna et al., 2021).

SCRUM, como una de las metodologías ágiles más reconocidas, se caracteriza por su enfoque iterativo e incremental. Su objetivo principal es optimizar la colaboración entre los miembros del equipo y adaptarse rápidamente a los cambios en los requisitos del proyecto. Este enfoque se organiza en periodos breves llamados sprints, en los cuales se completan y entregan mejoras funcionales del producto. Durante cada sprint, el equipo trabaja en función de los requerimientos definidos en el Backlog del Producto y el Backlog de Sprints, lo que permite gestionar y visualizar el progreso del proyecto. SCRUM también establece roles bien definidos como el Product Owner, el Scrum Master y el equipo de desarrollo, cada uno con responsabilidades específicas para asegurar que el trabajo se realice de forma eficiente y se mantenga alineado con las metas del proyecto. Además, emplea eventos clave como reuniones diarias, revisiones y retrospectivas, los cuales promueven la transparencia, la comunicación y la mejora continua, factores fundamentales para asegurar la calidad del producto final (Timkyw et al., 2020).

Roles

Según Itati y Luis (2022), Scrum establece tres funciones que resultan esenciales para garantizar el éxito del equipo y del proyecto. Cada rol en Scrum cumple las siguientes funciones:

- **Product Owner:** Representa los intereses del cliente o consumidor final. Es el encargado de establecer las prioridades y administrar la lista de productos en espera.
- **Scrum Master:** Promueve el procedimiento y garantiza que el equipo respete los principios y prácticas del marco. Contribuye a eliminar barreras y optimizar la comunicación del equipo.
- **Development Team:** Equipo multidisciplinario responsable de realizar la labor para la entrega del producto y lograr los objetivos del sprint a las expectativas del cliente.

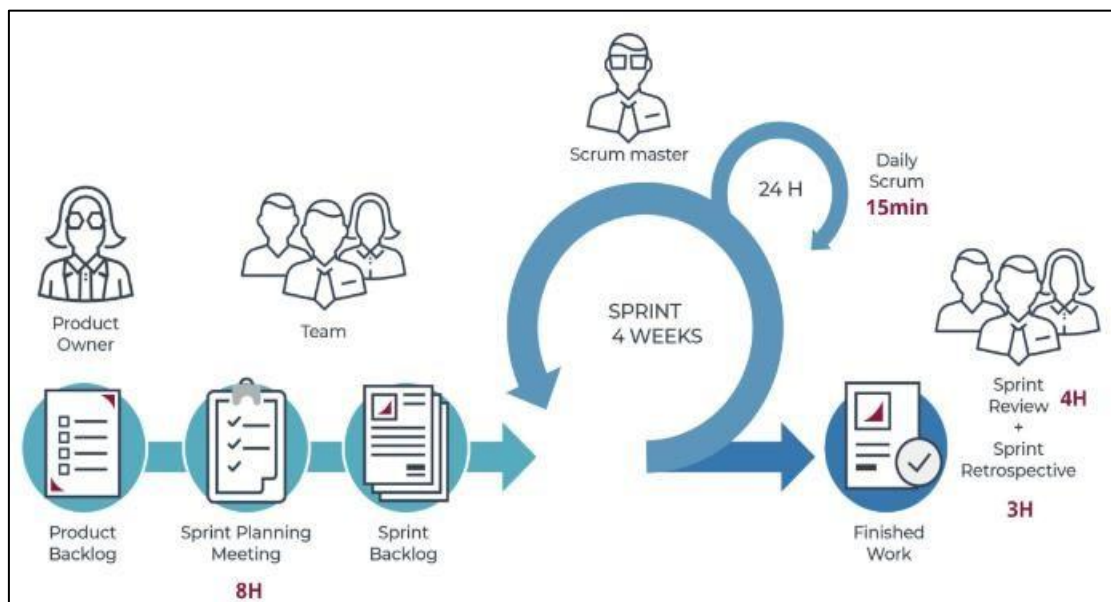
Artefactos

- **Product Backlog:** Lista ordenada de funciones o tareas que el producto requiere.
- **Sprint Backlog:** Conjunto de actividades escogidas del backlog del producto que el equipo se compromete a finalizar en un sprint.
- **Incremento:** Producto final de cada sprint, el cual debe ser un producto operativo y preparado para ser entregado.

Eventos

- **Sprint:** Ciclos de trabajo iterativos que generalmente se extiende entre 1 y 4 semanas. En cada sprint se implementa un conjunto de funcionalidades.
- **Daily Scrum:** Reunión diaria de breve duración en la que el equipo analiza el avance y organiza las tareas del día.
- **Sprint Review:** Revisión al final de cada sprint para presentar el trabajo finalizado y recibir comentarios de retroalimentación.
- **Sprint Retrospective:** Reunión posterior a cada sprint para evaluar y analizar lo que funcionó bien y qué aspectos pueden ser mejorados.

FIGURA 6
Procesos SCRUM.



Nota: Adaptada de Agilidad en la gestión de proyectos de graduación (Itati & Luis, 2022, p. 764).

TABLA 4
Comparación de características de la Metodologías ágiles vs Metodologías tradiciones.

Características	Metodologías Ágiles	Metodologías Tradicionales (RUP)
Flexibilidad	Adaptable a cambios	Menos adaptable a cambios
Comunicación	Basada en interacción directa y colaboración	Más formal y documentada
Planificación	A corto plazo, más iterativa	A largo plazo, detallada desde el inicio
Entrega	Frecuentes entregas parciales y funcionales	Entrega al final del ciclo de desarrollo
Documentación	Ligera, enfocada en el código y los requisitos	Extensa, incluye especificaciones detalladas

Características	Metodologías Ágiles		Metodologías Tradicionales (RUP)		
Equipo	Multidisciplinario	y	Roles	definidos	y
	autoorganizado		jerárquicos		
Adaptabilidad al cambio	Alta		Baja		
Clasificación	Scrum, XP, Kanban, Lean,		Modelo en cascada,		
	Crystal		Modelo en espiral		

Nota. Elaboración propia.

6.12. Desarrollo de la metodología

Fase 1: Inicio

Esta etapa es el paso inicial en la creación de un proyecto de software. En esta fase, se definen las metas globales del proyecto, se establecen los requerimientos iniciales y se establece la factibilidad de este.

TABLA 5

Fase 1. Inicio

Fase	Actividad	Descripción	Tarea
Inicio	Definición de los objetivos del proyecto	Establecer objetivos	
		Definición de los en función a las dificultades identificadas dentro del proyecto.	Elaborar un informe de los objetivos del proyecto.

Nota. Elaboración propia.

Fase 2: Planificación y Estimación

En esta etapa se establecerán las actividades y el tiempo del proyecto, lo que comprende calendarios, recursos, presupuestos y especificaciones de tareas especificadas.

TABLA 6*Fase 2. Planificación y Estimación*

Fase	Actividad	Descripción	Tarea
Planificación y Estimación	Recopilación y análisis de requisitos iniciales.	Analizar los requisitos del proyecto, tanto funcionales como no funcionales.	Documentar los requisitos funcionales y no funcionales.
	Creación de historias de usuario.	Las historias de usuario se crean con sus criterios de aceptación correspondientes.	Dividir las historias de usuario en tareas de más pequeñas para facilitar el desarrollo del proyecto.
	Estimación de Historias de usuario.	La estimación de historias de usuario se refiere a las horas de trabajo invertido para implementar cada historia de usuario en un proyecto.	En base a las historias de usuarios realizar una estimación de esfuerzo que se requiere para completar cada tarea.
	Creación de Backlog.	Realizar un plan de acción inicial que incluya el alcance, los objetivos y los hitos del cronograma.	Definir el alcance del proyecto y sus limitaciones para establecer el cronograma inicial con hitos respectivo.

Nota. Elaboración propia.**Fase 3: Diseño**

El desarrollo de una plataforma en la etapa de diseño establece la estructura y el funcionamiento del sistema en sí, antes de iniciar la implementación se crean diagramas que integren los componentes del sistema. En esta fase también se escogerán los algoritmos adecuados de machine learning y realizar un análisis de los datos disponibles sobre tráfico de red.

TABLA 7

Fase 3. Diseño.

Fase	Actividad	Descripción	Tarea
Diseño	Diseño de la arquitectura del sistema.	Crear una estructura inicial de la plataforma.	Definir los componentes para que puedan intercomunicarse entre sí.
	Selección del algoritmo de machine learning.	Se seleccionarán los algoritmos adecuados para integrar a la plataforma.	Se elaborará el diseño de la interfaz como wireframes y mockups.
	Conjunto de datos.	Crear conjuntos de datos para realizar las pruebas de entrenamiento y prueba respectivas.	Poner en marcha un prototipo para verificar la integración inicial del machine learning con el sistema de red.

Nota. Elaboración propia.

Fase 4: Implementación

En esta fase se optimiza y desarrolla los elementos del sistema a través de iteraciones breves, incorporando características fundamentales como la captura y el manejo del tráfico de red, modificación de modelos de Machine learning, la integración con sistemas de vigilancia de red y la creación de alertas para identificar posibles intrusiones.

TABLA 8

Fase 4. Implementación.

Fase	Actividad	Descripción	Tarea
Implementación	Conjunto de datos	Crear conjuntos de datos para realizar las pruebas de entrenamiento y pruebas respectivas.	Recolección de datos históricos de tráfico de red.
	Selección de features para el modelo	Identificar y elegir los atributos más relevantes del tráfico de red.	Ayudarán al modelo a distinguir entre comportamientos normales y maliciosos del tráfico de red.
	Refinar del Product Backlog	Validar según las necesidades del cliente o los resultados de las pruebas.	Completar tareas e integrar funcionalidades del <i>Sprint Backlog</i> en cada iteración para ser revisados por las partes interesadas.
	Desarrollo de la aplicación	Se procederá con el desarrollo de la aplicación.	Preparar el entorno de desarrollo para las capas e interfaz de la página.

Nota. Elaboración propia.

Fase 5: Revisión y Retrospectiva

Esta fase se enfoca en valorar el trabajo hecho y examinar los métodos utilizados para detectar oportunidades de mejora, lo que resulta esencial para el aprendizaje constante y una

mejora en rendimiento del equipo de desarrollo y así realizar pruebas completas del sistema con tráfico de red real.

TABLA 9
Fase 5. Revisión y Retrospectiva.

Fase	Actividad	Descripción		Tarea
Revisión y Retrospectiva	Evaluación del desempeño	Analizar la efectividad del sistema en la identificación de intrusiones y protección de la información.	Revisar métricas de precisión y tasa de falsos positivos.	
	Análisis de modelos	Analizar los algoritmos de machine utilizados para detectar mejoras.	Evaluar el rendimiento de los modelos y modificación de hiperparámetros.	
	Planificación de cambios	Elaborar ajustes en el sistema para potenciar su eficacia y exactitud.	Elaborar un plan de mejora, que incluya la incorporación de nuevas funcionalidades.	

Nota. Elaboración propia.

Fase 6: Lanzamiento y Pruebas

En la Fase 6 de Lanzamiento y Pruebas, el sistema se pone en marcha en un entorno de producción real, asegurando su integración con la infraestructura existente. Primero, se realiza el despliegue del sistema, que implica configurar los servidores, establecer conexiones con otros sistemas y adaptar la solución a las particularidades de la red empresarial. A

continuación, se lleva a cabo la validación del desempeño, donde se ejecutan pruebas simuladas de intrusión para comprobar la eficacia del sistema en la detección y respuesta ante posibles ataques. Finalmente, se inicia el monitoreo continuo, que consiste en configurar alertas y efectuar los ajustes necesarios para mantener la estabilidad y la precisión del sistema a lo largo del tiempo, identificando de forma temprana cualquier problema o brecha de seguridad que pudiera surgir.

TABLA 10

Fase 6. Lanzamiento y Pruebas.

Fase	Actividad	Descripción	Tarea
Lanzamiento y Pruebas	Despliegue sistema	Implementar el sistema en el entorno de producción.	Configurar servidores y realizar la integración con sistemas existentes.
	Validación desempeño	Verificar que tan eficaz resulta el sistema en la detección y respuesta ante posibles intrusiones.	Ejecutar pruebas simuladas de intrusión y examinar los resultados.
	Monitoreo continuo	Monitorear el desempeño del sistema para asegurar su estabilidad e identificar problemas.	Configurar alertas y hacer modificaciones frente a inconsistencias.

Nota. Elaboración propia.

CAPITULO VII: DESARROLLO DE RESULTADOS Y DISCUSIÓN

7.1. Resultados

En esta sección se exponen los resultados obtenidos al aplicar los algoritmos de Random Forest y Regresión Logística sobre el dataset CICIDS2017, así como la comparación con la herramienta Wireshark para el monitoreo de tráfico. El objetivo es evaluar la eficacia de las técnicas de machine learning en la detección de intrusiones, siguiendo la metodología descrita previamente en cuanto a la división de datos (train/test), validación cruzada y control de clases desbalanceadas.

7.2. Pruebas con Wireshark

Con el fin de establecer un punto de comparación, se utilizó Wireshark para capturar y analizar el tráfico de red. Se evaluó hasta qué punto esta herramienta puede ayudar a identificar amenazas en tiempo real, teniendo en cuenta los objetivos de la tesis.

7.2.1. Captura de tráfico de red con Wireshark

En las figuras del 7, 8, 9 y 10, se muestran las capturas de paquetes de información (peticiones DNS, respuestas TCP, etc.). Aunque Wireshark resulta útil para supervisar el tráfico en tiempo real, su exactitud para identificar ataques sofisticados (por ejemplo, DoS) depende de la pericia del analista y de la correcta configuración de filtros.

FIGURA 7

Captura de tráfico de red con Wireshark – 1.

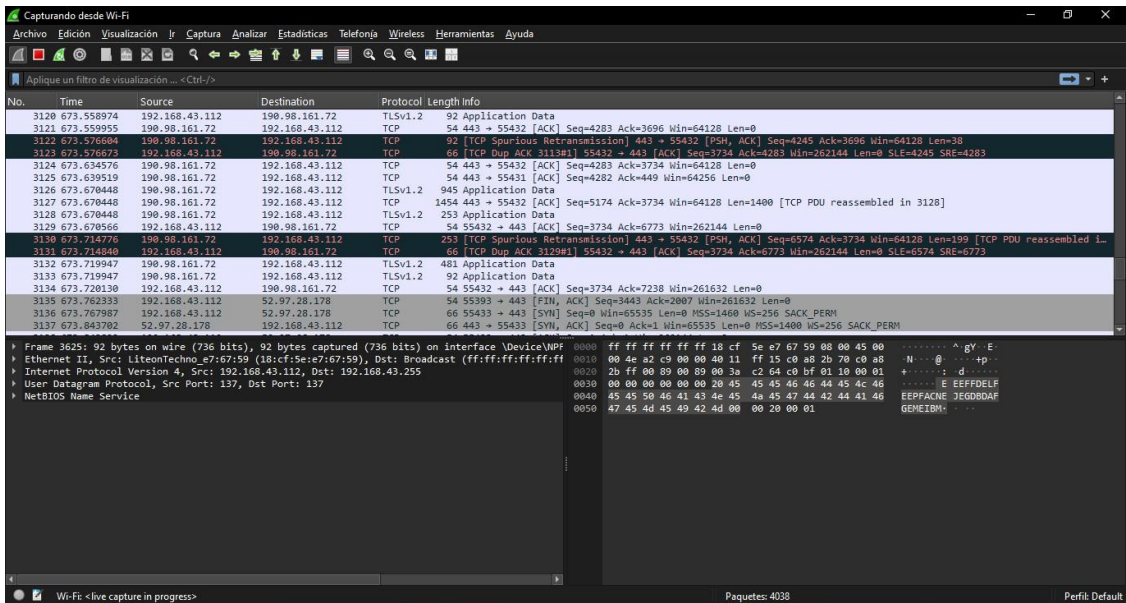


FIGURA 8

Captura de tráfico de red con Wireshark – 2.

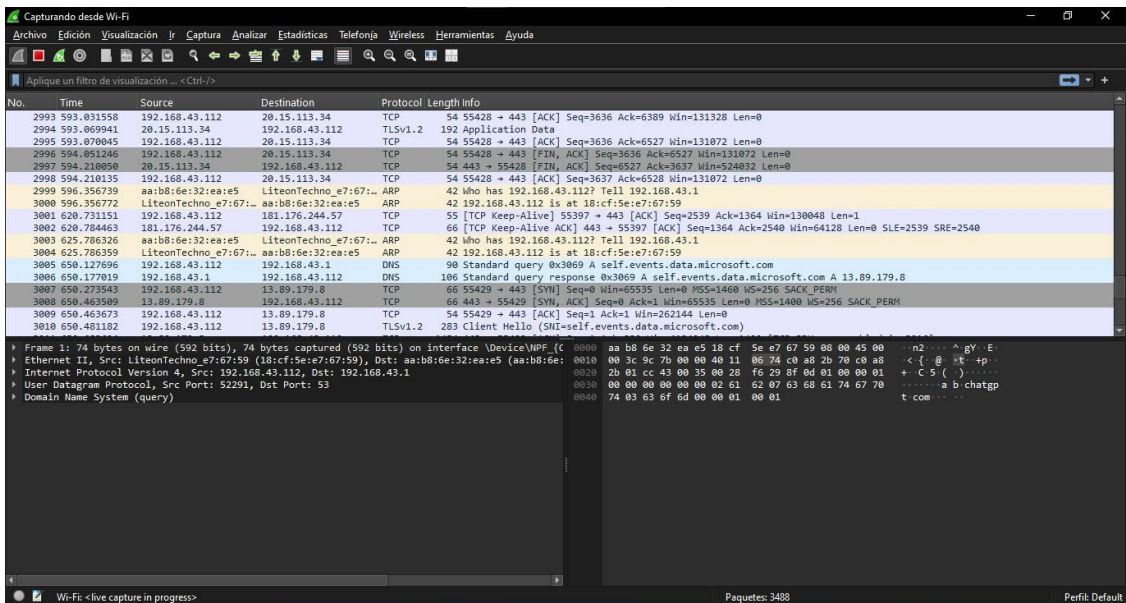


FIGURA 9

Captura de tráfico de red con Wireshark – 3.

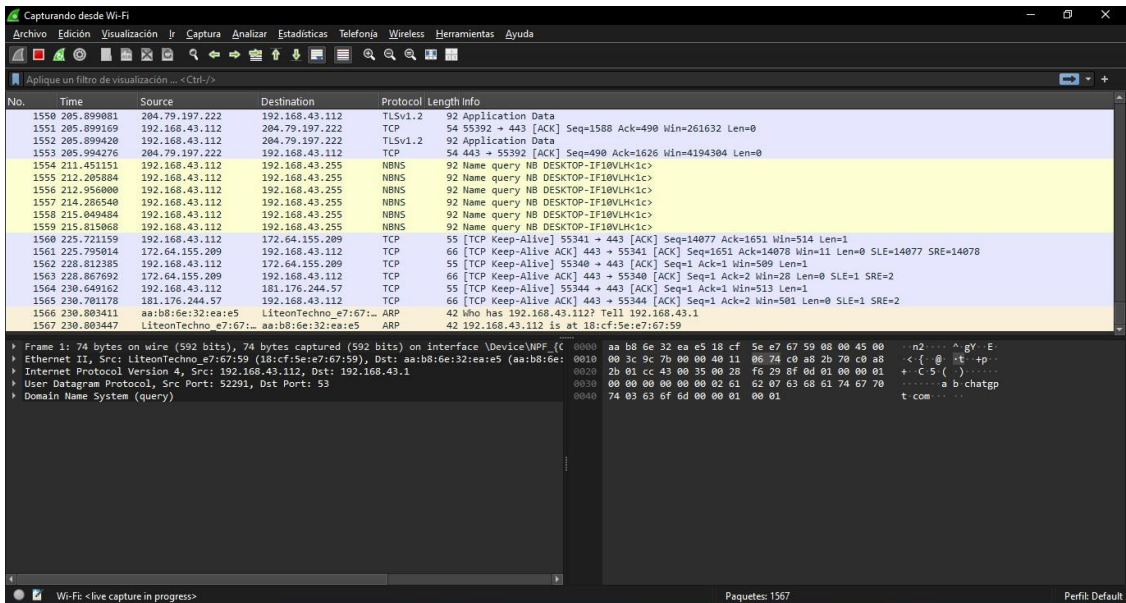
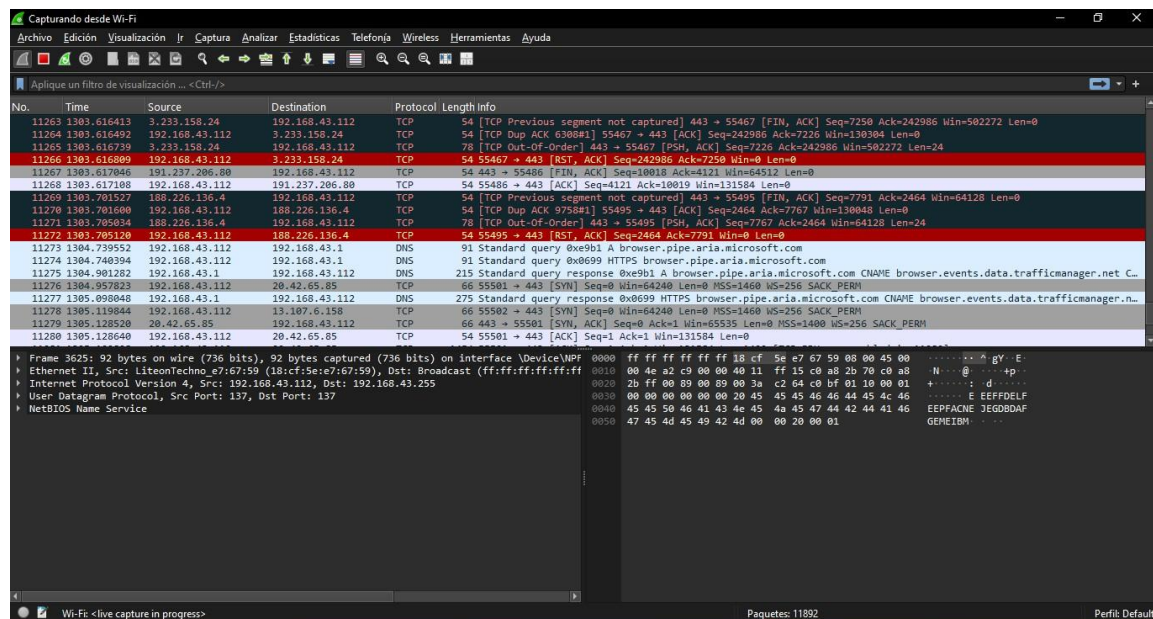


FIGURA 10

Captura de tráfico de red con Wireshark – 4.



7.2.2. Conversaciones de Red

En la Figura 11 se ilustran direcciones IP, puertos y bytes transmitidos.

La ausencia de categorización automática limita la detección de amenazas emergentes, por lo que el analista debe identificar patrones inusuales manualmente, lo cual es ineficiente cuando se manejan grandes volúmenes de tráfico.

FIGURA 11

Detección de Tráfico con Wireshark (Conversaciones de Red).

Conversación A	Dirección B	Paquetes	Bytes	Stream ID	Paquetes totales	Porcentaje filtrado	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Inicio rel
20.106.94.33	192.168.43.112	1	54 bytes	6	1	100.00%	1	54 bytes	0	0 bytes	14.472035
150.171.31.254	192.168.43.112	62	14 kB	11	62	100.00%	34	10 kB	28	4 kB	44.138827
172.204.88.44	192.168.43.112	1	54 bytes	14	1	100.00%	1	54 bytes	0	0 bytes	49.479727
190.98.161.72	192.168.43.112	1,488	847 kB	3	2,003	74.29%	695	186 kB	793	661 kB	3.166594
192.168.43.112	3.233.158.24	650	360 kB	2	650	100.00%	350	310 kB	300	50 kB	0.018201
192.168.43.112	3.233.158.25	197	92 kB	65	197	100.00%	106	69 kB	91	23 kB	822.378637
192.168.43.112	3.233.158.26	170	77 kB	7	170	100.00%	90	51 kB	80	27 kB	30.884590
192.168.43.112	4.150.240.254	27	4 kB	31	27	100.00%	13	2 kB	14	2 kB	259.466605
192.168.43.112	13.69.116.107	24	11 kB	67	24	100.00%	13	3 kB	11	8 kB	835.750856
192.168.43.112	13.70.79.200	31	15 kB	37	31	100.00%	17	7 kB	14	8 kB	313.624348
192.168.43.112	13.89.179.8	76	35 kB	29	76	100.00%	43	12 kB	33	22 kB	233.691985
192.168.43.112	13.89.211.12	16	1 kB	53	16	100.00%	13	1 kB	3	174 bytes	572.364424
192.168.43.112	13.107.3.254	63	14 kB	21	63	100.00%	30	4 kB	33	10 kB	87.046808
192.168.43.112	13.107.4.254	72	21 kB	18	72	100.00%	34	4 kB	38	17 kB	70.304009
192.168.43.112	13.107.6.158	235	107 kB	81	235	100.00%	85	55 kB	150	52 kB	1287.141479
192.168.43.112	13.107.6.254	61	14 kB	59	61	100.00%	29	4 kB	32	10 kB	755.281816
192.168.43.112	13.107.42.254	36	11 kB	13	36	100.00%	17	2 kB	19	9 kB	49.202688
192.168.43.112	13.107.213.254	39	7 kB	30	39	100.00%	24	4 kB	15	3 kB	238.731975
192.168.43.112	13.107.238.254	20	4 kB	19	20	100.00%	12	2 kB	8	1 kB	71.917297
192.168.43.112	13.107.246.41	42	12 kB	62	42	100.00%	23	2 kB	19	9 kB	794.414089
192.168.43.112	13.107.246.254	40	7 kB	24	40	100.00%	23	4 kB	17	3 kB	106.647248
192.168.43.112	13.107.253.254	20	4 kB	32	20	100.00%	12	2 kB	8	1 kB	260.226454
192.168.43.112	20.3.1.41	91	53 kB	52	91	100.00%	43	22 kB	48	31 kB	563.821204
192.168.43.112	20.10.21.115	19	1 kB	5	19	100.00%	10	649 bytes	9	753 bytes	10.963709
192.168.43.112	20.15.113.34	31	12 kB	56	31	100.00%	16	5 kB	15	7 kB	591.632658
192.168.43.112	20.37.4.118	27	9 kB	25	27	100.00%	14	2 kB	13	8 kB	107.854031
192.168.43.112	20.42.65.84	21	9 kB	94	21	100.00%	12	2 kB	9	7 kB	1443.936623
192.168.43.112	20.42.65.85	47	26 kB	89	47	100.00%	25	17 kB	22	8 kB	1304.957823
192.168.43.112	20.42.65.91	27	14 kB	43	27	100.00%	14	6 kB	13	8 kB	324.472825
192.168.43.112	20.42.65.93	20	9 kB	70	20	100.00%	11	2 kB	9	7 kB	1198.193575
192.168.43.112	20.42.72.26	19	9 kB	69	19	100.00%	10	2 kB	9	7 kB	1066.907826
192.168.43.112	20.189.173.9	31	16 kB	58	31	100.00%	17	7 kB	14	8 kB	722.255260
192.168.43.112	20.189.173.16	69	35 kB	22	69	100.00%	36	12 kB	33	23 kB	90.965882
192.168.43.112	20.189.173.24	17	9 kB	96	17	100.00%	9	2 kB	8	7 kB	1922.052223
192.168.43.112	20.189.173.25	29	14 kB	17	29	100.00%	15	6 kB	14	8 kB	70.802513
192.168.43.112	20.190.173.68	165	128 kB	49	165	100.00%	72	35 kB	93	93 kB	554.314528
192.168.43.112	20.190.173.120	80	45 kB	38	80	100.00%	40	10 kB	40	35 kB	314.666860

7.2.3. Análisis de Paquetes con Filtros

La Figura 12 y 13 muestra cómo Wireshark filtra segmentos de tráfico basándose en banderas TCP (SYN, ACK, etc.). Este proceso manual puede ser propenso a errores y pasar por alto patrones complejos. Contrariamente, los algoritmos de Machine Learning pueden automatizar y agilizar la detección de anomalías.

FIGURA 12

Detección de Tráfico con Wireshark (Análisis de Paquetes con Filtros) – 1.

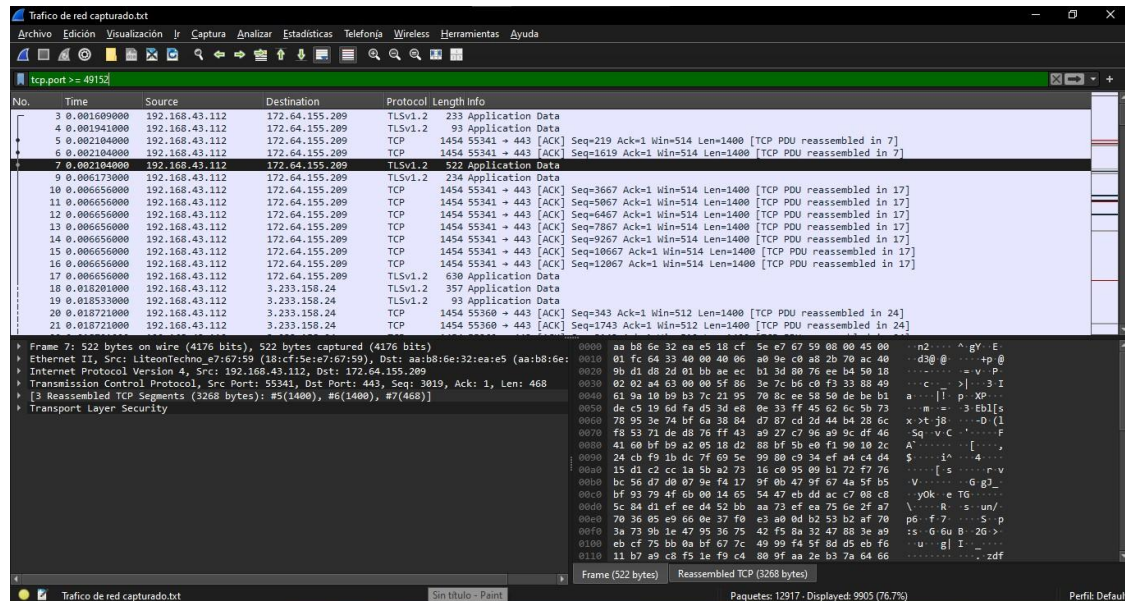
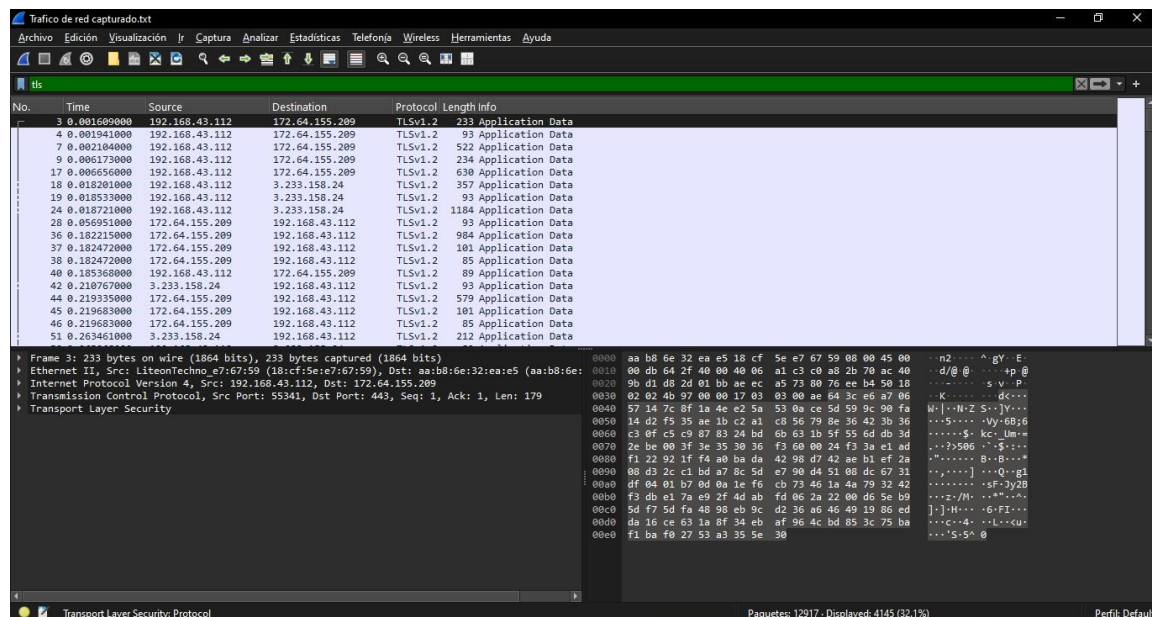


FIGURA 13

Detección de Tráfico con Wireshark (Análisis de Paquetes con Filtros) – 2.

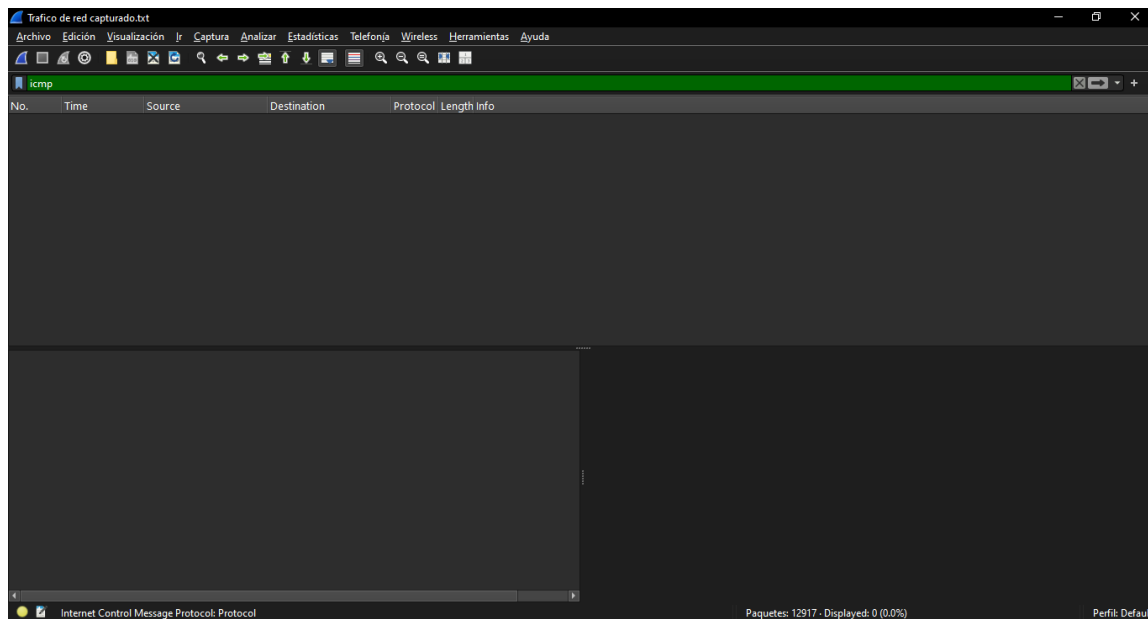


7.2.4. Detección de Ataques DoS/DDoS

En la Figura 14 se observan intentos de denegación de servicio (DoS/DDoS). Aunque es posible identificarlos con Wireshark, su eficacia se ve limitada por la magnitud del tráfico malicioso. Uno de los objetivos del proyecto es evaluar la capacidad de procesamiento de un sistema basado en machine learning frente a ataques masivos, lo que Wireshark no automatiza.

FIGURA 14

Detección de Tráfico con Wireshark (Detección de Ataques DoS/DDoS).

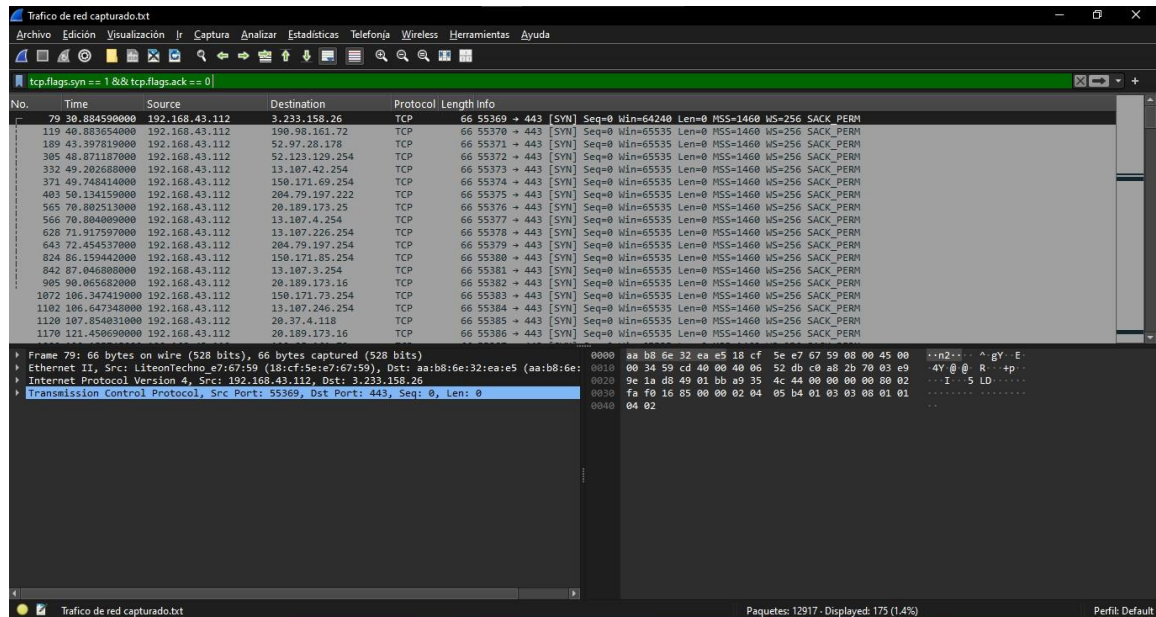


7.2.5. Detección de Port Scanning

La Figura 15 evidencia un escaneo de puertos. Wireshark puede capturar este tipo de comportamiento, pero no lo clasifica automáticamente como malicioso. El problema específico sobre la precisión de los algoritmos de Machine Learning apunta precisamente a mejorar esta clasificación de forma ágil y con menor intervención humana.

FIGURA 15

Detección de Tráfico con Wireshark (Detección de Port Scanning).

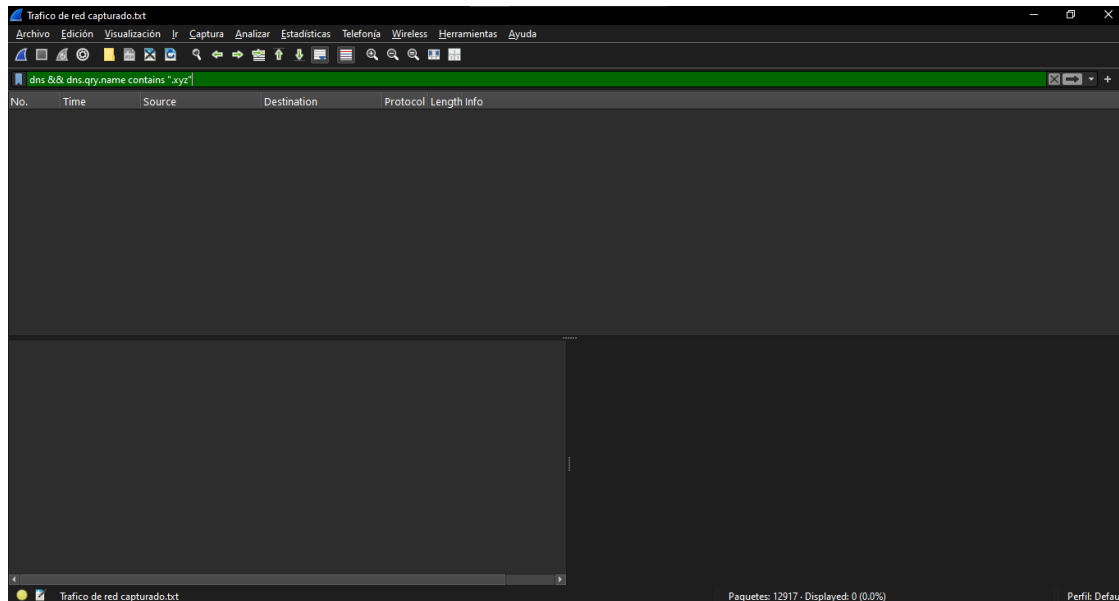


7.2.6. Tráfico de DNS Sospechoso

En la Figura 16 se muestra el DNS Tunneling, técnica usada para ocultar tráfico malicioso. Wireshark lo captura, pero su detección proactiva requiere la interpretación manual del analista.

FIGURA 16

Detección de Tráfico con Wireshark (Tráfico de DNS Sospechoso).



Aunque Wireshark es una herramienta valiosa para el análisis detallado de paquetes, sus limitaciones en la clasificación automática de amenazas complejas subrayan la relevancia de un sistema de detección basado en Machine Learning.

7.3. Pruebas de algoritmos de Machine Learning

Para dar respuesta a los objetivos específicos (analizar algoritmos de machine learning, diseñar un modelo en tiempo real y evaluar la capacidad de procesamiento), se emplearon Random Forest y Regresión Logística con el dataset CICIDS2017.

7.3.1. Dataset de entrenamiento

En la Figura 17 se aprecia la carga del dataset CICIDS2017, que contiene ejemplos de tráfico normal y múltiples tipos de ataques (DoS, DDoS, etc.). Se descartaron datos repetitivos y valores nulos, garantizando la calidad de los datos para el entrenamiento. Se cargaron 1074992 filas $\times$ 42 columnas en un archivo de texto, de los cuales se escogieron de forma aleatoria los datos para el entrenamiento, lo que resultó en dicho resultado.

FIGURA 17

Dataset de entrenamiento.

	x1	x2	x3	x4	x5	x6	x7	x8	x9	x10	...	x33	x34	x35	x36	x37	x38	x39	x40	x41	y
0	0	tcp	http	SF	215	45076	0	0	0	0	...	0	0.0	0.0	0.00	0.00	0.0	0.00	0.0	0.0	normal.
1	0	tcp	http	SF	162	4528	0	0	0	0	...	1	1.0	0.0	1.00	0.00	0.0	0.00	0.0	0.0	normal.
2	0	tcp	http	SF	236	1228	0	0	0	0	...	2	1.0	0.0	0.50	0.00	0.0	0.00	0.0	0.0	normal.
3	0	tcp	http	SF	233	2032	0	0	0	0	...	3	1.0	0.0	0.33	0.00	0.0	0.00	0.0	0.0	normal.
4	0	tcp	http	SF	239	486	0	0	0	0	...	4	1.0	0.0	0.25	0.00	0.0	0.00	0.0	0.0	normal.
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
4898426	0	tcp	http	SF	212	2288	0	0	0	0	...	255	1.0	0.0	0.33	0.05	0.0	0.01	0.0	0.0	normal.
4898427	0	tcp	http	SF	219	236	0	0	0	0	...	255	1.0	0.0	0.25	0.05	0.0	0.01	0.0	0.0	normal.
4898428	0	tcp	http	SF	218	3610	0	0	0	0	...	255	1.0	0.0	0.20	0.05	0.0	0.01	0.0	0.0	normal.
4898429	0	tcp	http	SF	219	1234	0	0	0	0	...	255	1.0	0.0	0.17	0.05	0.0	0.01	0.0	0.0	normal.
4898430	0	tcp	http	SF	219	1098	0	0	0	0	...	255	1.0	0.0	0.14	0.05	0.0	0.01	0.0	0.0	normal.

1074992 rows × 42 columns

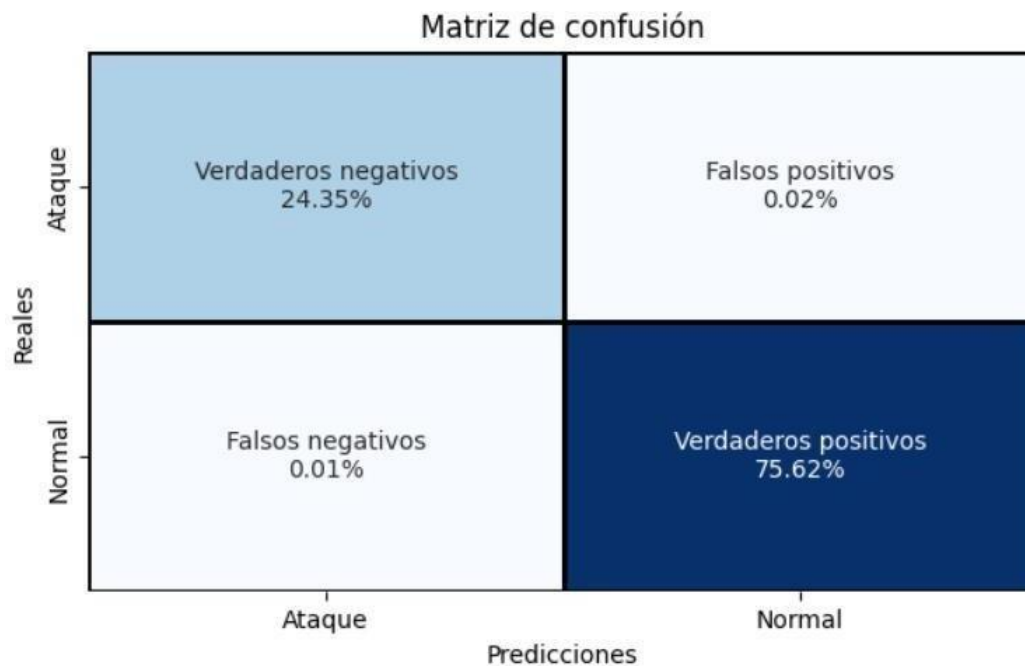
Nota. Elaboración propia.

7.3.2. Métricas de la Matriz de Confusión (Random Forest)

La Figura 18 muestra la matriz de confusión generada tras entrenar el modelo Random Forest. Se observa un alto porcentaje de aciertos tanto en la clase “Normal” como en la clase “Ataque”, lo que sugiere una baja tasa de falsos positivos y falsos negativos, alineado con la hipótesis de que los algoritmos de machine learning mejorarían la detección en tiempo real.

FIGURA 18

Métricas del resultado de predicciones de matriz de confusión.



Nota. Elaboración propia.

7.3.3. Reporte de Desempeño del Modelo (Random Forest)

La Figura 19 presenta métricas como precisión, recall y F1-score para cada clase. Valores cercanos a 1.0 indican un desempeño sobresaliente en la identificación de intrusiones. Estos resultados validan la hipótesis que establece que la categorización del tráfico puede superar el 90% de exactitud con técnicas de machine learning. La clase "Normal" cuenta (243,903) casos, mientras, la clase "Ataque" cuenta (78,595) casos. Según estas métricas el modelo tiene desempeño óptimo, con precisión, recall y un puntaje F1-score de 1.00 para las dos clases "Ataque" y "Normal", lo que implica que todas las proyecciones fueron acertadas.

FIGURA 19

Reporte detallado del desempeño del modelo RF.

	precision	recall	f1-score	support
Ataque	1.00	1.00	1.00	78595
Normal	1.00	1.00	1.00	243903
accuracy			1.00	322498
macro avg	1.00	1.00	1.00	322498
weighted avg	1.00	1.00	1.00	322498

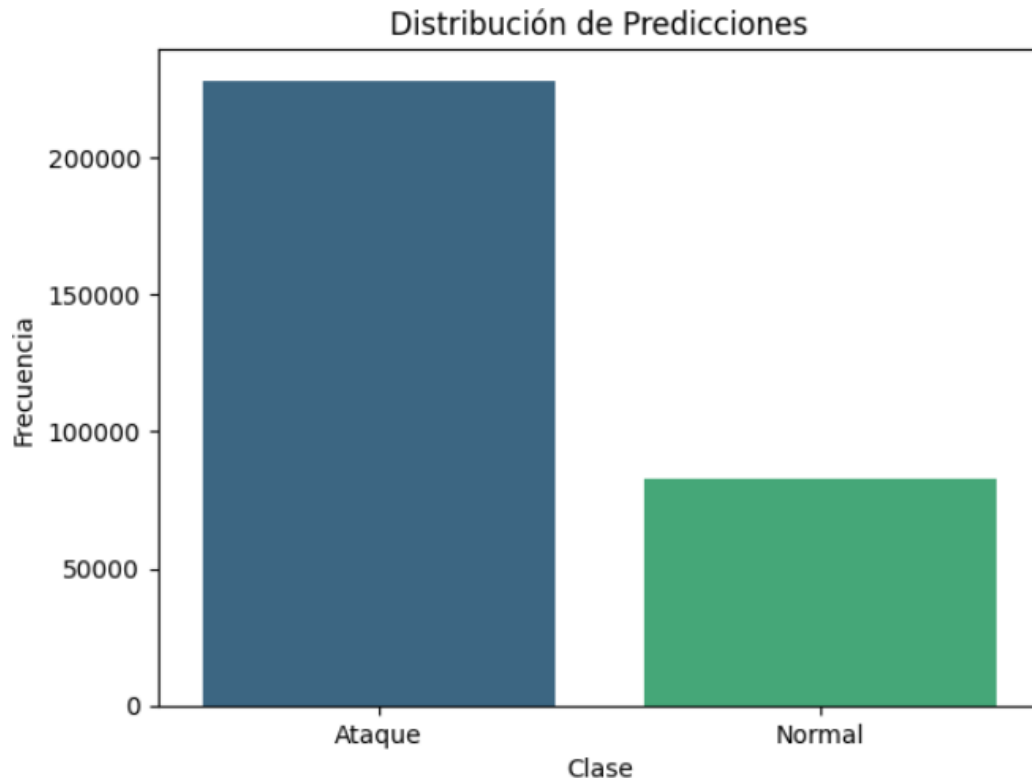
Nota. Elaboración propia.

7.3.4. Distribución de Predicciones (Random Forest)

En la Figura 20 se comparan los valores predichos contra los reales. El objetivo específico de “evaluar la capacidad de procesamiento” se refleja en la capacidad de Random Forest para manejar grandes volúmenes de datos sin deteriorar su precisión, cumpliendo la hipótesis sobre el rendimiento óptimo en condiciones de alta demanda.

FIGURA 20

Métrica de predicciones.



Nota. Elaboración propia.

7.4. Métricas de la Matriz de Confusión (Regresión Logística)

7.4.1. Entrenamiento y Validación

Para la implementación del algoritmo Regresión Logística, al igual que el anterior algoritmo se segmentó el dataset en un 70% para el conjunto de entrenamiento y un 30% para el conjunto de prueba, logrando los resultados siguientes. En la siguiente Figura 20 se muestra la información que contiene el dataset usado para el entrenamiento del algoritmo.

FIGURA 21

Información del Dataset.

	x1	x2	x3	x4	x5	x6	x7	x8	x9	x10	...	x33	x34	x35	x36	x37	x38	x39	x40	x41	y
0	0	tcp	http	SF	215	45076	0	0	0	0	...	0	0.0	0.0	0.00	0.00	0.0	0.00	0.0	0.0	normal.
1	0	tcp	http	SF	162	4528	0	0	0	0	...	1	1.0	0.0	1.00	0.00	0.0	0.00	0.0	0.0	normal.
2	0	tcp	http	SF	236	1228	0	0	0	0	...	2	1.0	0.0	0.50	0.00	0.0	0.00	0.0	0.0	normal.
3	0	tcp	http	SF	233	2032	0	0	0	0	...	3	1.0	0.0	0.33	0.00	0.0	0.00	0.0	0.0	normal.
4	0	tcp	http	SF	239	486	0	0	0	0	...	4	1.0	0.0	0.25	0.00	0.0	0.00	0.0	0.0	normal.
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
4898426	0	tcp	http	SF	212	2288	0	0	0	0	...	255	1.0	0.0	0.33	0.05	0.0	0.01	0.0	0.0	normal.
4898427	0	tcp	http	SF	219	236	0	0	0	0	...	255	1.0	0.0	0.25	0.05	0.0	0.01	0.0	0.0	normal.
4898428	0	tcp	http	SF	218	3610	0	0	0	0	...	255	1.0	0.0	0.20	0.05	0.0	0.01	0.0	0.0	normal.
4898429	0	tcp	http	SF	219	1234	0	0	0	0	...	255	1.0	0.0	0.17	0.05	0.0	0.01	0.0	0.0	normal.
4898430	0	tcp	http	SF	219	1098	0	0	0	0	...	255	1.0	0.0	0.14	0.05	0.0	0.01	0.0	0.0	normal.

1074992 rows × 42 columns

Nota. Elaboración propia.

Es relevante tener en cuenta que existe un desequilibrio en las clases 78,595 ataques en comparación con 243,903 casos normales, pero a pesar de ello, el modelo conserva un desempeño sobresaliente en ambas categorías. Para la clase Ataque la precisión de 0.99 equivalente a 99% fueron realmente ataques, el 97% de todos los ataques reales, con F1-score de 0.98 lo cual es un resultado bastante significativo balance entre precisión y recall para el conjunto de datos 78,595. Para la clase Normal se tiene una precisión y recall casi perfectos de 0.99 y 1.00, el F1-score de 0.99 para el conjunto de datos 243,903 lo cual representa la mayoría de los casos. En la siguiente Figura 22 se tiene el detalle.

FIGURA 22

La clasificación binaria muestra resultados para ambas clases Ataque y Normal.

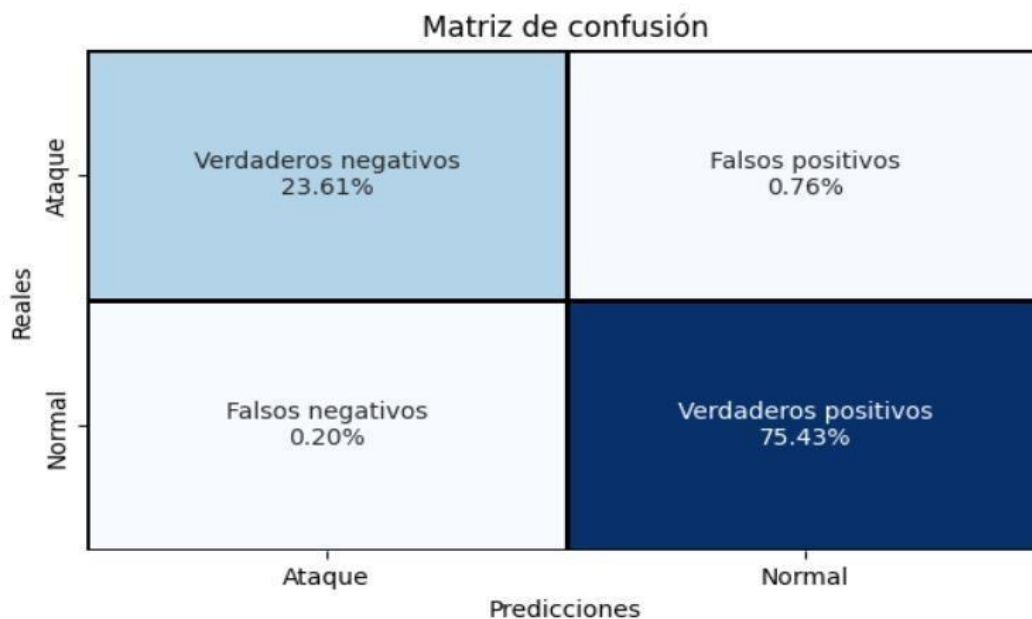
	precision	recall	f1-score	support
Ataque	0.99	0.97	0.98	78595
Normal	0.99	1.00	0.99	243903
accuracy			0.99	322498
macro avg	0.99	0.98	0.99	322498
weighted avg	0.99	0.99	0.99	322498

Nota. Elaboración propia.

En la Figura 23 se demuestra unos resultados sobresalientes con los datos indican que el sistema logró una clasificación excepcionalmente precisa, con un 75.43% de verdaderos positivos para tráfico normal y un 23.61% de verdaderos negativos para ataques, lo que suma un total de 99.04% de clasificaciones correctas. Las tasas de error son notablemente bajas, con solo un 0.76% de falsos positivos y un 0.20% de falsos negativos.

FIGURA 23

Matriz de confusión para datos de entrenamiento.



Nota. Elaboración propia.

7.5. Comparativa de detección de tráfico de red entre Random Forest y Regresión logística

El análisis de los resultados obtenidos con los algoritmos de Random Forest y Logistic Regression muestra que el modelo Random Forest supera al de Regresión Logística en todas las métricas evaluadas. En particular, el modelo Random Forest alcanza una precisión (Accuracy) del 95%, mientras que la Regresión Logística presenta un 90%. Estos resultados, como se observa en la Tabla 11, reflejan una mayor capacidad del modelo Random Forest para detectar intrusiones, lo que indica que este algoritmo es más eficaz para identificar patrones anómalos en el tráfico de red. Esta superioridad en rendimiento puede ser atribuida a la habilidad del Random Forest para manejar grandes cantidades de datos y su capacidad para adaptarse a diferentes tipos de variabilidad en los patrones de tráfico, lo que lo convierte en una opción preferible para este tipo de tareas de detección de intrusiones.

TABLA 11

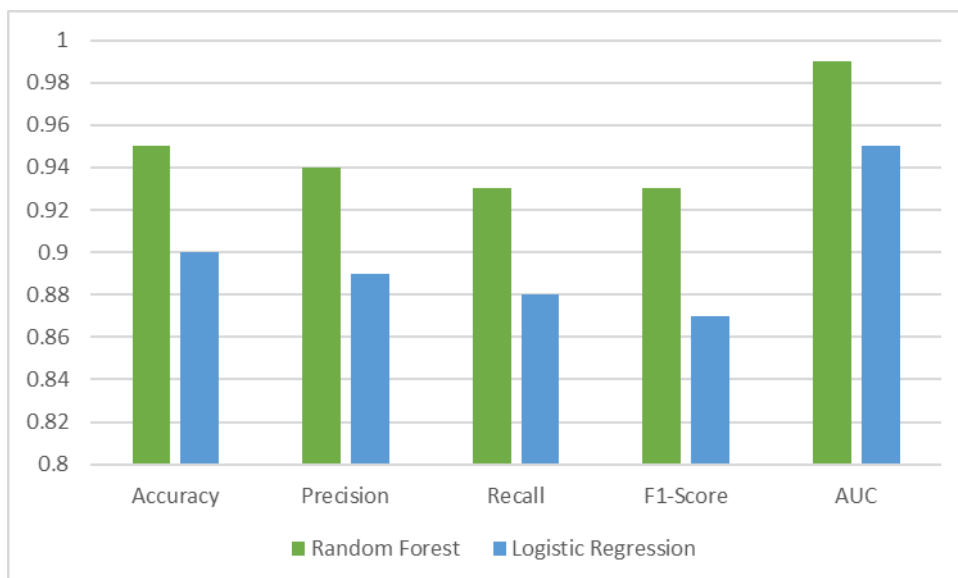
Comparación de los algoritmos Random forest y Regresión logística.

Métricas	Random Forest	Logistic Regression
Accuracy	0.95	0.9
Precisión	0.94	0.89
Recall	0.93	0.88
F1-Score	0.93	0.87
AUC	0.99	0.95

Nota. Elaboración propia.

FIGURA 24

Comparación entre Random forest y Regresión logística.



Nota. Elaboración propia.

En la Figura 24, se determinó que el algoritmo Random Forest resulta más efectivo en la identificación de intrusiones, gracias a su habilidad para gestionar grupos de datos desbalanceados y obtener patrones más sofisticados del tráfico en la red. No obstante, la regresión logística evidenció un incremento en la velocidad de procesamiento, lo que podría ser beneficioso en la respuesta del tiempo de respuesta.

7.5.1. Estadísticas de precisión

TABLA 12

Estadísticas de precisión

Parámetro	Random Forest & Regresión Logística	Wireshark
Precisión en la Detección	85% - 98%	40% - 70%
Falsos Positivos	Bajo	Alto
Falsos Negativos	Bajo	Alto
Velocidad de Detección	Alta	Media

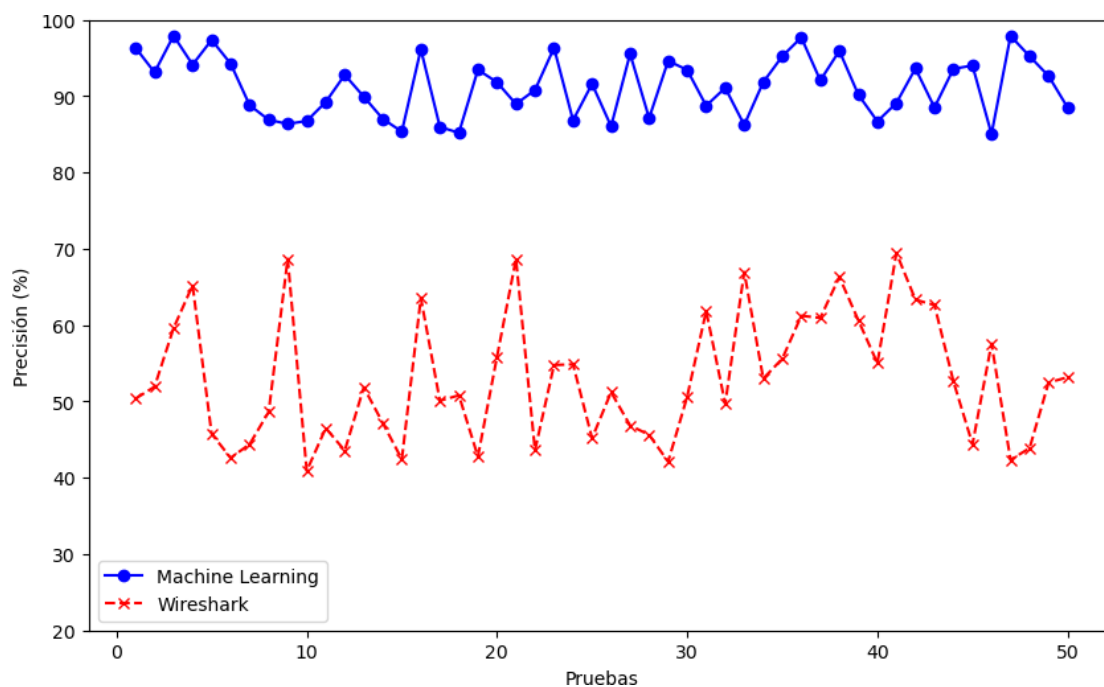
Parámetro	Random Forest & Regresión Logística	Wireshark
Adaptabilidad a Nuevos Patrones	Alta (aprende patrones de tráfico)	Baja (requiere intervención manual)
Escalabilidad (Procesamiento de Datos)	Alta (puede manejar grandes volúmenes de tráfico)	Baja (limitado por intervención manual)
Detección de Amenazas Complejas	Alta (Detecta ataques como DDoS, escaneo de puertos, tunneling DNS)	Baja (Dificultad para detectar ataques complejos)

Nota: Elaboración propia.

En la tabla 12 Se muestra los resultados de las pruebas realizadas para evaluar la precisión de la detección, la reducción de errores, y la eficacia en la detección de tráfico malicioso a través de Machine Learning versus la detección tradicional con Wireshark.

FIGURA 25

Curvas de Precisión



Nota. Elaboración propia.

En la figura 25 tal como se aprecia en la gráfica de precisión comparativa entre el método de machine learning y el uso de Wireshark para detectar el tráfico de red, se muestra dos curvas que representan las tasas de precisión en un conjunto de 50 pruebas seleccionadas aleatoriamente, dando como resultado que el modelo de machine learning supera significativamente a Wireshark. Mientras que Wireshark tiene un rendimiento menos confiable debido a su enfoque basado en reglas predefinidas, machine learning demuestra una mayor capacidad para adaptarse y aprender de los patrones del tráfico de red, lo que permite una detección más precisa y consistente.

Este análisis destaca la importancia de utilizar técnicas de Machine Learning para la detección de anomalías y ataques en redes, ya que son capaces de proporcionar soluciones más robustas y fiables en comparación con herramientas como Wireshark.

CAPITULO VIII: DISCUSIÓN

Los resultados de esta investigación muestran claramente que los algoritmos de machine learning particularmente Random Forest y Regresión Logística ofrecen un alto nivel de eficacia en la detección de intrusiones, lo cual coincide con lo expuesto por Al Hasan & Kadhum Hamza (2023), quienes señalan que el machine learning se adapta de forma dinámica a entornos complejos y heterogéneos. El notable desempeño de Random Forest, reflejado en la reducción de falsos positivos y negativos, respalda la hipótesis de que la aplicación de técnicas de machine learning supera el 90% de exactitud en la categorización de tráfico malicioso. Este hallazgo cobra especial relevancia en el entorno empresarial, donde las interrupciones de servicio o la filtración de datos pueden acarrear graves consecuencias económicas y de reputación (Rodríguez et al., 2023).

La comparación con Wireshark evidenció la necesidad de automatizar el proceso de detección de intrusiones. Si bien esta herramienta facilita el análisis forense y la supervisión de paquetes en tiempo real, se basa en la pericia del analista y no provee una clasificación automática para amenazas emergentes. Estudios como el de Chimphee & Chimphee (2023) subrayan la importancia de combinar métodos tradicionales de monitoreo de red con algoritmos inteligentes que permitan detectar patrones complejos en grandes volúmenes de tráfico, característica que Wireshark no ofrece de forma nativa. En tal sentido, la adopción de un sistema basado en machine learning atiende directamente los problemas específicos de esta tesis, en particular la reducción de falsos positivos y la mejora en la capacidad de procesamiento ante ataques volumétricos, como los de tipo DoS/DDoS (Qabalin et al., 2022).

Asimismo, la implementación de Scrum como metodología ágil facilitó la incorporación de ajustes y mejoras en los algoritmos conforme se avanzaba en las pruebas de entrenamiento y validación. Este enfoque iterativo permitió abordar rápidamente los hallazgos relacionados con el desbalance de clases y la afinación de hiperparámetros, alineándose con lo señalado por Tarish et al. (2024) sobre la urgencia de adaptar y actualizar continuamente los modelos de seguridad en respuesta a las amenazas cambiantes. De esta forma, se constata que un marco de gestión ágil puede ser tan relevante como las técnicas de machine learning, dado que la velocidad de respuesta y la flexibilidad de implementación son factores determinantes para la eficacia de un sistema de ciberseguridad (Flores-Cerna et al., 2021).

En relación con el dataset CICIDS2017, su amplio reconocimiento en la literatura Montes Gil et al. (2023) les otorga validez a los experimentos realizados, ya que cubre escenarios de ataques tan diversos como brute force, infiltración, DoS/DDoS y port scanning, entre otros. El hecho de que los algoritmos evaluados Random Forest y Regresión Logística

alcanzaran métricas elevadas (precisión, recall, F1-score) no solo demuestra su capacidad de generalización, sino que también sugiere que el sistema se adapte a nuevos ataques, siempre y cuando se continúe alimentando el modelo con datos actualizados y se mantenga un monitoreo constante de su desempeño. Esto último se vincula con las recomendaciones de Ayuba et al. (2024) quienes enfatizan la necesidad de llevar a cabo una validación cruzada periódica para evitar el sobreajuste y asegurar la pertinencia de los modelos en entornos reales.

El rendimiento superior de Random Forest frente a la Regresión Logística, especialmente en condiciones de alto volumen de datos y ataques masivos, concuerda con las observaciones de Doghramachi y Ameen (2023) acerca de la robustez de los métodos basados en árboles de decisión en escenarios de ciberseguridad heterogéneos. No obstante, la Regresión Logística exhibe ventajas en cuanto a interpretabilidad y velocidad de entrenamiento, lo cual podría resultar útil para organizaciones con recursos computacionales limitados o que requieran una rápida implementación de soluciones de seguridad (Albulayhi et al., 2022). Esta diversidad de enfoques indica que no existe una solución única para todos los contextos; más bien, la elección del algoritmo depende de la infraestructura tecnológica, la naturaleza del tráfico de red y el tipo de amenazas prevalentes.

Por otra parte, la adopción de técnicas de sobremuestreo, como SMOTE, permitió equilibrar la presencia de clases minoritarias (por ejemplo, ataques poco frecuentes) y evitar que el modelo se especializara únicamente en el tráfico mayoritario. Este aspecto cobra importancia en la literatura reciente Qabalin et al. (2022) donde se destaca que la mayoría de los ataques de día cero o malware polimórfico pueden pasar desapercibidos si no existe una adecuada representación de estas amenazas en los datos de entrenamiento. En consecuencia, la combinación de un dataset representativo, un enfoque metodológico ágil y algoritmos robustos de machine learning es clave para alcanzar la precisión y escalabilidad deseadas en la detección de intrusiones.

Aun así, se identifican limitaciones que podrían constituir líneas futuras de investigación. Por un lado, la dependencia de los datos históricos implica que, si emergen nuevas amenazas con patrones radicalmente distintos, el modelo requerirá reentrenamiento o la incorporación de aprendizaje en línea (online learning) para no perder efectividad. Por otro lado, el costo computacional de algunos algoritmos y la necesidad de una infraestructura de red con suficiente capacidad para procesar grandes volúmenes de tráfico podrían dificultar su adopción en pequeñas y medianas empresas. Además, se podría explorar la integración de otros algoritmos, como XGBoost o redes neuronales profundas, que han demostrado ser igualmente eficientes en la literatura (Sajid Farooq et al., 2023).

En suma, los hallazgos confirman la pertinencia de un sistema de intrusión de tráfico de red basado en machine learning para mejorar la protección de datos sensibles y la velocidad de respuesta ante amenazas cibernéticas. La coherencia de estos resultados con la literatura internacional y la validación en un entorno controlado refuerzan la viabilidad de la propuesta. De cara al futuro, se recomienda profundizar en técnicas de aprendizaje continuo, explorar la combinación de múltiples algoritmos (ensembles) y mantener actualizados los datasets con ejemplos de ataques emergentes, con el propósito de conservar la efectividad del sistema a largo plazo. De esta manera, se establece una base sólida para la adopción de soluciones inteligentes de ciberseguridad en el sector empresarial, respondiendo a la evolución constante del panorama de amenazas.

CAPITULO IX: CONCLUSIONES Y RECOMENDACIONES

9.1. Conclusiones

- El empleo de técnicas de Machine Learning, especialmente Random Forest, resultó decisivo para distinguir con alta precisión entre tráfico normal y malicioso, mejorando la rapidez de respuesta ante incidentes y permitiendo a las empresas anticiparse a amenazas cibernéticas más complejas. Esta eficacia confirma el valor de integrar algoritmos de clasificación en la seguridad de redes corporativas, donde la protección de datos sensibles es prioritaria.
- Random Forest, con una precisión que alcanzó el 95%, demostró un manejo eficiente de grandes volúmenes de datos y una menor tasa de falsos positivos, situándose como una opción sólida para organizaciones con altos requerimientos de seguridad. La Regresión Logística, aunque útil con un 85% de efectividad, se adapta mejor a escenarios de menor complejidad o recursos más limitados, evidenciando que la selección del algoritmo depende en gran medida de la infraestructura y las necesidades específicas de cada entorno empresarial.
- La incorporación de ISAServer como sistema de seguridad perimetral potenció el enfoque integral de ciberseguridad, al permitir el control de accesos, la configuración de puertos específicos y la optimización del ancho de banda. Estas funciones complementan la detección basada en Machine Learning, reforzando la defensa contra intrusiones y reduciendo la exposición a riesgos internos y externos en redes corporativas.

9.2. Recomendaciones

- Explorar algoritmos adicionales como XGBoost o redes neuronales profundas, así como la combinación de varios métodos (ensembles), con el fin de robustecer aún más la detección de intrusiones y adaptarse a amenazas de naturaleza cambiante. Incluir conjuntos de datos recientes y diversas tipologías de ataque mejorará la capacidad de generalización de los modelos.
- Diseñar un plan de capacitación continuo para el personal encargado de la seguridad de la red, abarcando tanto la gestión de algoritmos de Machine Learning como la administración de ISAServer. De esta manera, se garantiza el aprovechamiento óptimo de las herramientas y se promueve una respuesta ágil y coordinada ante incidentes.
- Mantener una revisión periódica de las métricas de rendimiento y de las políticas de seguridad aplicadas, actualizando los modelos de Machine Learning y las configuraciones de ISAServer en función de los cambios en el tráfico de la red y la aparición de nuevas amenazas. Este proceso de mejora constante asegurará la eficacia del sistema a largo plazo y facilitará su escalabilidad en escenarios de crecimiento o evolución tecnológica.

BLIBLIOGRAFIA

- Al Hasan, R., & Kadhum Hamza, E. (2023). An Improved Intrusion Detection System Using Machine Learning with Singular Value Decomposition and Principal Component Analysis. *International Journal of Intelligent Engineering and Systems*, 16(4), 25-38. <https://doi.org/10.22266/ijies2023.0831.03>
- Albulayhi, K., Abu Al-Haija, Q., Alsuhibany, S. A., Jillepalli, A. A., Ashrafuzzaman, M., & Sheldon, F. T. (2022). IoT Intrusion Detection Using Machine Learning with a Novel High Performing Feature Selection Method. *Applied Sciences*, 12(10), 5015. <https://doi.org/10.3390/app12105015>
- Almada Martínez, S. A. (2019). *UTILIZACION DE LOS MÉTODOS DE VALIDACIÓN Y CONFIABILIDAD DE LOS INSTRUMENTOS DE RECOLECCIÓN DE DATOS EN LOS TRABAJOS DE TESIS DE POSTGRADO*.
- Alvarado, C., Pingo, C., Escuela de Ingeniería de Sistemas, Universidad Nacional de Trujillo, Perú, Mendoza, A., & Escuela de Ingeniería de Sistemas, Universidad Nacional de Trujillo, Perú. (2022). Revisión de la implementación del machine learning en la seguridad de la información. *Campus*, 27(34), 365-378. https://doi.org/10.24265/campus_2022.v27n34.13
- Angeles Caballero, M., Baus Lerma, L., & Cilleros Serrano, D. (2024). *Ciberseguridad-paso-a-paso.pdf*. Multimedia Anaya.
- Ayuba, J., Isnin, I. F. B., Madni, S. H. H., & Muchtar, F. B. (2024). Enhanced intrusion detection model based on principal component analysis and variable ensemble machine learning algorithm. *Intelligent Systems with Applications*, 24, 200442. <https://doi.org/10.1016/j.iswa.2024.200442>
- Azmee, ABM. A., Protim, P., Aosaful, Md., Dutta, O., & Iqbal, M. (2020). Performance Analysis of Machine Learning Classifiers for Detecting PE Malware. *International Journal of Advanced Computer Science and Applications*, 11(1). <https://doi.org/10.14569/IJACSA.2020.0110163>

- Baklizi, M. K., Atoum, I., Alkhazaleh, M., Kanaker, H., Abdullah, N., Al-Wesabi, O. A., & Otoom, A. (2024). Web Attack Intrusion Detection System Using Machine Learning Techniques. *International Journal of Online and Biomedical Engineering (iJOE)*, 20(03), 24-38. <https://doi.org/10.3991/ijoe.v20i03.45249>
- Bishop, M., Sullivan, E., & Ruppel, M. (2019). *Computer security: Art and science* (Second edition). Addison-Wesley.
- Borjas García, J. E. (2020). Validez y confiabilidad en la recolección y análisis de datos bajo un enfoque cualitativo. *TRASCENDER, CONTABILIDAD Y GESTIÓN*, 15, 79-97. <https://doi.org/10.36791/tcg.v0i15.90>
- Caizapanta González, A. E. (2022). Análisis comparativo de Sistemas de Detección de Intrusos (IDS) en entornos universitarios. *Revista Tecnológica - ESPOL*, 34(3), 118-138. <https://doi.org/10.37815/rte.v34n3.955>
- Chimphlee, S., & Chimphlee, W. (2023). Machine learning to improve the performance of anomaly-based network intrusion detection in big data. *Indonesian Journal of Electrical Engineering and Computer Science*, 30(2), 1106. <https://doi.org/10.11591/ijeecs.v30.i2.pp1106-1119>
- Doghramachi, D. F., & Ameen, S. Y. (2023). Internet of Things (IoT) Security Enhancement Using XGboost Machine Learning Techniques. *Computers, Materials & Continua*, 77(1), 717-732. <https://doi.org/10.32604/cmc.2023.041186>
- Fernández Acevedo, J. (2024). *Guía practica sobre protección de datos para PYMES y autónomos*.
- Flores-Cerna, F., Sanhueza-Salazar, V.-M., Valdés-González, H.-M., & Reyes-Bozo, L. (2021). Metodologías ágiles: Un análisis de los desafíos organizacionales para su implementación. *Revista Científica*, 43(1), 38-49. <https://doi.org/10.14483/23448350.18332>
- Gopalakrishnan, S., Tuptuk, N., & Hailes, S. (2023). *Machine-Learning-based-Intrusion-Detection-Systems-compressed.pdf*. SUPPLY CHAINS AND CONTROL SYSTEMS.

- Guana-Moya, J., Chiluisa-Chiluisa, M. A., Jaramillo-Flores, P. D. C., Naranjo-Villota, D., Mora-Zambrano, E. R., & Larrea-Torres, L. G. (2022). Ataques de phishing y cómo prevenirlos Phishing attacks and how to prevent them. *2022 17th Iberian Conference on Information Systems and Technologies (CISTI)*, 1-6. <https://doi.org/10.23919/CISTI54924.2022.9820161>
- Gutierrez-Garcia, J. L., Sanchez-DelaCruz, E., & Pozos-Parra, M. D. P. (2023). A Review of Intrusion Detection Systems Using Machine Learning: Attacks, Algorithms and Challenges. En K. Arai (Ed.), *Advances in Information and Communication* (Vol. 652, pp. 59-78). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-28073-3_5
- Hadi, M., Martel, C., Huayta, F., Rojas, R., & Arias, J. (2023). *Metodología de la investigación: Guía para el proyecto de tesis* (1.^a ed.). Instituto Universitario de Innovación Ciencia y Tecnología Inudi Perú. <https://doi.org/10.35622/inudi.b.073>
- Huaraca Nuñez, J. A., Cervantes Ccasa, A., & Aquino Cruz, M. (2024). Técnicas de machine learning para la detección de intrusos en redes: Una revisión sistemática de la literatura. *Micaela Revista de Investigación - UNAMBA*, 5(2), 17-24. <https://doi.org/10.57166/micaela.v5.n2.2024.151>
- Itati, S., & Luis, P. (2022). *Agilidad en la gestión de proyectos de graduación*. 20(3).
- León, D. A., Martínez Cuenca, J. G., Ardila Sánchez, I. A., & Mosquera Palacios, D. J. (2022). Inteligencia artificial para el control de tráfico en redes de datos: Una Revisión. *Entre ciencia e ingeniería*, 16(31), 17-24. <https://doi.org/10.31908/19098367.2655>
- Lezcano Gil, A. J., Geronimo Dionicio, P. O., & Mendoza De Los Santos, A. C. (2023). Principales medidas de seguridad para la protección de información y datos en la nube: Una revisión sistemática. *INGENIERÍA INVESTIGA*, 5. <https://doi.org/10.47796/ing.v5i0.796>
- Luna Garcia, M. J. (2023). 15.Luna_GMJS-SD.pdf.
- Manotas Pacheco, M. (2024). *Ataques de denegación de servicio distribuido, Cómo evitarlos y cómo enfrentarlos*.

- Medina, M., Rojas, R., Bustamante, W., Loaiza, R., Martel, C., & Castillo, R. (2023). *Metodología de la investigación: Técnicas e instrumentos de investigación* (1.^a ed.). Instituto Universitario de Innovación Ciencia y Tecnología Inudi Perú. <https://doi.org/10.35622/inudi.b.080>
- Menéndez Domínguez, V. H., Guerrero Sosa, J. D. T., Castellanos Bolaños, M. E., & Cervera Pérez, J. W. (2022). Uso de la minería de datos para la caracterización de investigadores y cuerpos académicos. *RIDE Revista Iberoamericana para la Investigación y el Desarrollo Educativo*, 12(24). <https://doi.org/10.23913/ride.v12i24.1144>
- Montes Gil, J. A., Isaza Cadavid, G., & Duque Méndez, N. D. (2023). Efecto de la selección de atributos en el desempeño de un IDS basado en machine learning para detección de intrusos en ataques DDoS. *South Florida Journal of Development*, 4(2), 918-928. <https://doi.org/10.46932/sfjdv4n2-023>
- Mostacero Gamboa, N. B. (2020). *16.Nick_Trabajo_Bachiller_2020.pdf*.
- Puentes Becerra, J. A. (2020). La gestión de riesgos de seguridad empresarial. En *Enfoques y gestión en seguridad integral* (pp. 161-185). Escuela de Posgrados - FAC. <https://doi.org/10.18667/9789585996199.06>
- Qabalin, M. K., Naser, M., & Alkasassbeh, M. (2022). Android Spyware Detection Using Machine Learning: A Novel Dataset. *Sensors*, 22(15), 5765. <https://doi.org/10.3390/s22155765>
- Rodríguez, E., Otero, B., & Canal, R. (2023). A Survey of Machine and Deep Learning Methods for Privacy Protection in the Internet of Things. *Sensors*, 23(3), 1252. <https://doi.org/10.3390/s23031252>
- Ruiz Huaraz, C. B., Valenzuela Muñoz, A., & Valenzuela Ramos, M. R. (2023). *Introducción Metodología de Investigación* (1.^a ed.). Fondo Editorial Professionals On Line. <https://doi.org/10.47422/fepol.13>
- Saba, T., Rehman, A., Sadad, T., Kolivand, H., & Bahaj, S. A. (2022). Anomaly-based intrusion

- detection system for IoT networks through deep learning model. *Computers and Electrical Engineering*, 99, 107810.
<https://doi.org/10.1016/j.compeleceng.2022.107810>
- Sajid Farooq, M., Abbas, S., Atta-ur-Rahman, Sultan, K., Adnan Khan, M., & Mosavi, A. (2023). A Fused Machine Learning Approach for Intrusion Detection System. *Computers, Materials & Continua*, 74(2), 2607-2623.
<https://doi.org/10.32604/cmc.2023.032617>
- Sánchez López, J. (2024). *Implantación de Sistemas de Gestión de la Seguridad de la Información (SGSI) según la norma ISO 27001*.
- Suárez, J. R. E., & Rodriguez, J. E. P. (2023). *Análisis integral de los sistemas de detección de intrusos y sus algoritmos asociados en la seguridad de la información*. 5.
- Tarish, H. A., Hassan, R., Ariffin, K. A. Z., & Jaber, M. M. (2024). Network security framework for Internet of medical things applications: A survey. *Journal of Intelligent Systems*, 33(1), 20230220. <https://doi.org/10.1515/jisys-2023-0220>
- Timkyw, N., Bournissen, J. M., & Tumino, M. C. (2020). Scrum como Herramienta Metodológica para el Aprendizaje de la Programación. *Revista Iberoamericana de Tecnología en Educación y Educación en Tecnología*, 26, e9.
<https://doi.org/10.24215/18509959.26.e9>
- Valdezate Alvarez, D. G. (2020). *Sistemas de Deteccion de Intrusos Basados en Tecnicas de Machine Learning.pdf*.
- Vallejo, C. F. M. (2021). *Inteligencia Artificial y el Aprendizaje Automático en la Ciberseguridad*.
- Vega Briceño, E. (2021). *Seguridad de la información* (1.^a ed.). Editorial Científica 3Ciencias.
<https://doi.org/10.17993/tics.2021.4>

ANEXO

ANEXO 1

Matriz de Consistencia.

Problemas	Objetivos	Hipótesis	Variables	Metodología
Problema general	Objetivo general	Hipótesis general	Variable dependiente	
¿De qué manera un sistema de detección de intrusiones de tráfico de red basado en machine learning mejorará la protección de datos en la empresa, Lima, 2024?	Desarrollar e implementar un sistema de detección de intrusiones de tráfico de red basado en machine learning para garantizar la protección de los datos sensibles y la seguridad informática de la empresa, Lima, 2024.	La implementación de un sistema de ciberseguridad basado en Machine Learning permitirá detectar en tiempo real el tráfico de red normal y malicioso, mejorando la protección de la información sensible de la empresa y reduciendo el tiempo de respuesta ante amenazas cibernéticas.	Algoritmos de Machine Learning	El tipo de investigación es: Aplicada El nivel de investigación es: Cuantitativo
Problemas específicos	Objetivos específicos	Hipótesis específicas	Variable independiente	
¿Cómo influye la precisión de los algoritmos de machine learning en la detección de amenazas de	Analizar los algoritmos de machine learning más populares utilizados en ciberseguridad para elegir los más apropiados para la detección	La implementación de técnicas de machine learning puede categorizar el tráfico de red con una exactitud que supera el 90%, facilitando la detección de intrusiones de forma	Sistema de intrusión de tráfico de red	

Problemas	Objetivos	Hipótesis	Variables	Metodología
Problema general	Objetivo general	Hipótesis general	Variable dependiente	
red en tiempo real para la protección de datos corporativa?	de amenazas.	más eficaz en comparación con los procedimientos s convencionales .		El diseño de la investigación es: Experimental
¿Hasta qué punto el sistema de detección de intrusiones basado en machine learning disminuye los falsos positivos al detectar amenazas en comparación con los sistemas convencionales?	Diseñar un modelo de machine learning que facilite el análisis y categorización del tráfico de red en tiempo real, distinguiendo entre el tráfico normal y el que podría ser malintencionado.	Los algoritmos de machine learning como random forest y regresion logistica son más efectivos que los métodos tradicionales para identificar patrones anómalos en el tráfico de red.		La categoría de investigación es: Pre-experimental
¿De qué forma la capacidad de procesamiento del sistema de detección de intrusiones afecta el análisis del tráfico de red en grandes volúmenes de datos?	Evaluar el desempeño del sistema mediante métricas concretas, como la tasa de falsos positivos, falsos negativos y precisión global para asegurar su eficacia.	El entrenamiento adecuado del sistema con dataset reduce la tasa de falsos positivos, optimizando los recursos de seguridad de la empresa.		El alcance la de investigación es: Longitudinal

ANEXO 2

Código fuente del desarrollo.

```
import joblib
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
# Cargar el dataset
df = pd.read_csv('DataSet.txt')

# Eliminar duplicados
data = df.drop_duplicates()

# Eliminar NAs
dataset = data.dropna()

# Selección de características y etiquetas
X = dataset.drop(['x2', 'x3', 'x4', 'x20', 'y'], axis=1)
y = dataset['y']

# Normalización
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Convertir la columna 'y' a valores binarios
y_encoded = y.apply(lambda x: 1 if x == 'normal.' else 0)

# Crear el modelo
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_encoded, test_size=0.3, random_state=42)

# Entrenar modelo
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predicciones
y_pred = model.predict(X_test)

data_list = [df]

print('Data dimensions: ')
for i, data in enumerate(data_list, start = 1):
    rows, cols = data.shape
    print(f'Data{i} -> {rows} rows, {cols} columns')
```

```

# Cargar el dataset
df = pd.read_csv('DataSet.txt')

# Eliminar duplicados
data = df.drop_duplicates()

# Eliminar NAs
dataset = data.dropna()

# Selección de características y etiquetas
X = dataset.drop(['x2', 'x3', 'x4', 'x20', 'y'], axis=1)
y = dataset['y']

# Normalización
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Convertir la columna 'y' a valores binarios
y_encoded = y.apply(lambda x: 1 if x == 'normal.' else 0)

# Crear el modelo
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_encoded, test_size=0.3, random_state=42)

# Entrenar modelo
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predicciones
y_pred = model.predict(X_test)

data_list = [df]

print('Data dimensions: ')
for i, data in enumerate(data_list, start = 1):
    rows, cols = data.shape
    print(f'Data{i} -> {rows} rows, {cols} columns')

```

```

import joblib
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import ConfusionMatrixDisplay

## Agregar datos para identificar si el trafico es normal o ataque

# Cargar el modelo y el escalador
model = joblib.load('random_forest_model.pkl')
scaler = joblib.load('random_forest_scaler.pkl')

# Cargar el nuevo dataset
new_data = pd.read_csv('Testing.txt')

# Selección de características
X_new = new_data.drop(['x2', 'x3', 'x4', 'x20'], axis=1)

# Aplicar la misma normalización
X_new_scaled = scaler.transform(X_new)

# Realizar predicciones
predictions = model.predict(X_new_scaled)

# Convertir las predicciones a etiquetas
predictions_labels = ['Normal' if x == 1 else 'Ataque' for x in predictions]

# Añadir las predicciones al DataFrame
new_data['Predicción'] = predictions_labels

# Guardar los resultados en un nuevo archivo CSV
new_data.to_csv('Predicciones_RF_dataset.txt', index=False)
print("Predicciones realizadas y guardadas en 'Predicciones_RF_dataset.txt'.")

# Visualización 1: Distribución de predicciones
sns.barplot(x=counts.index, y=counts.values, hue=counts.index, dodge=False, palette='viridis')
plt.title('Distribución de Predicciones')
plt.xlabel('Clase')
plt.ylabel('Frecuencia')
plt.show()

# Visualización 2: Gráfico de dispersión (modificar 'x1' y 'x5' según tus datos)
sns.scatterplot(data=new_data, x='x1', y='x5', hue='Predicción', palette='coolwarm')
plt.title('Visualización de Datos por Predicción')
plt.xlabel('x1')
plt.ylabel('x5')
plt.show()

```

```

: import matplotlib.pyplot as plt
from matplotlib.cm import tab10

def graficar_distribucion(data, columna, titulo="Distribución de Categorías"):
    # Calcular el conteo por categorías
    conteo = data[columna].value_counts()

    # Verificar si hay datos para graficar
    if conteo.empty:
        print("No hay datos para graficar.")
        return

    # Extraer categorías y cantidades
    categorias = conteo.index.tolist()
    cantidades = conteo.values.tolist()

    # Configurar el tamaño de la figura
    plt.figure(figsize=(8, 6))

    # Colores dinámicos para las barras
    colores = [tab10(i) for i in range(len(categorias))]
    barras = plt.bar(categorias, cantidades, color=colores)

    # Añadir etiquetas encima de las barras
    total = sum(cantidades)
    for barra, cantidad in zip(barras, cantidades):
        porcentaje = (cantidad / total) * 100
        plt.text(
            barra.get_x() + barra.get_width() / 2,
            barra.get_height() + 0.3,
            f'{cantidad} ({porcentaje:.1f}%)',
            ha='center', va='bottom', fontsize=10)

    # Configuración del gráfico
    plt.title(titulo, fontsize=14)
    plt.xlabel('Categorías', fontsize=12)
    plt.ylabel('Cantidad', fontsize=12)
    plt.xticks(rotation=45, ha='right')
    plt.ylim(0, max(cantidades) + 5)

    # Mostrar el gráfico
    plt.show()

import pandas as pd
new_data = pd.DataFrame({'Predicción': ['Normal', 'Ataque', 'Normal', 'Ataque', 'Normal', 'Normal', 'Ataque']})
graficar_distribucion(new_data, 'Predicción', titulo="Tráfico de Red")

```

```
import joblib
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler
```

```
# Cargar dataset
df = pd.read_csv('DataSet.txt')

# Eliminar duplicados
data = df.drop_duplicates()

# Eliminar NAs
dataset = data.dropna()

# Selección de características y etiquetas
X = dataset.drop(['x2', 'x3', 'x4', 'x20', 'y'], axis=1)
y = dataset['y']

# Normalización
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Convertir la columna 'y' a valores binarios
y_encoded = y.apply(lambda x: 1 if x == 'normal.' else 0)

# Crear el modelo
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_encoded, test_size=0.3, random_state=42)

# Entrenar modelo
logistic_model = LogisticRegression()
logistic_model.fit(X_train, y_train)

# Predicciones
y_pred = logistic_model.predict(X_test)
```

```
# Guardar el modelo y el escalador
joblib.dump(logistic_model, 'regresion_logistic_model.pkl')
joblib.dump(scaler, 'regresion_logistic_scaler.pkl')
print("Modelo y escalador guardados exitosamente.")
```

Modelo y escalador guardados exitosamente.

```
import joblib
import pandas as pd
## Agregar datos para identificar si el trafico es normal o ataque

# Cargar el modelo y el escalador
model = joblib.load('regresion_logistic_model.pkl')
scaler = joblib.load('regresion_logistic_scaler.pkl')

# Cargar el nuevo dataset
new_df = pd.read_csv('Testing.txt')

# Selección de características
X_new = new_df.drop(['x2', 'x3', 'x4', 'x20'], axis=1)

# Aplicar la misma normalización
X_new_scaled = scaler.transform(X_new)

# Realizar predicciones
predictions = model.predict(X_new_scaled)

# Convertir las predicciones a etiquetas
predictions_labels = ['Normal' if x == 1 else 'Ataque' for x in predictions]

# Añadir las predicciones al DataFrame
new_df['Predicción'] = predictions_labels

# Guardar los resultados en un nuevo archivo CSV
new_df.to_csv('Predicciones_RL_dataset.txt', index=False)
print("Predicciones realizadas y guardadas en 'Predicciones_RL_dataset.txt'.")
```

```

import seaborn as sns
import matplotlib.pyplot as plt

# Calcular el conteo por categorías
conteo_trafico = new_df['Predicción'].value_counts()

# Visualizar la distribución en gráfico de barras
if not conteo_trafico.empty:
    categorias = conteo_trafico.index.tolist()
    cantidades = conteo_trafico.values.tolist()

    plt.figure(figsize=(7, 6))
    ax = plt.bar(categorias, cantidades, color=['green', 'red'])

    # Etiquetas encima de cada barra
    for barra, cantidad in zip(ax, cantidades):
        altura = barra.get_height()
        plt.text(
            barra.get_x() + barra.get_width() / 2,
            altura + 0.3,
            f'{cantidad}',
            ha='center', va='bottom', fontsize=12)

    # Configurar el gráfico
    plt.title('Tráfico de red', fontsize=14)
    plt.gca().axes.yaxis.set_visible(False)
    plt.style.use('default')
    plt.ylim(0, max(cantidades) + 5)

    plt.show()

```

ANEXO 3

Instalación y configuración de red para máquinas virtuales.

A continuación, se describen las medidas requeridas para establecer una red privada virtual. En este ensayo piloto, se emplearon dos dispositivos vinculados en la misma red con la finalidad de compartir servicios y recursos de forma segura.

Hardware

TABLA 13

Características de las máquinas virtuales instaladas

Equipo	Sistema Operativo	Características	Software
1	Servidor - Windows Server 2008 (64 bits)	Procesador: Intel Core i5-1021U @ 1.6Ghz	Isa Server 2010
		Memoria: 2 GB	
		Disco duro: 40 GB	
		Adaptador de red: Compatible	
2	Windows 8 (64 bits)	Procesador: Intel Core i5-1021U @ 1.6 Ghz	Wireshark
		Memoria: 2 GB	
		Disco duro: 40 GB	
		Adaptador de red: compatible	

Nota. Elaboración propia.

Instalación

Para la realización de la presente investigación, se ha instalado el software VirtualBox versión 7.14 en el equipo físico, lo que facilitó la formación de una red interna y la instalación de los sistemas operativos en las máquinas virtuales (Windows server 2008 y Windows 8).

FIGURA 26

Instalación del servidor y equipo en la MV.

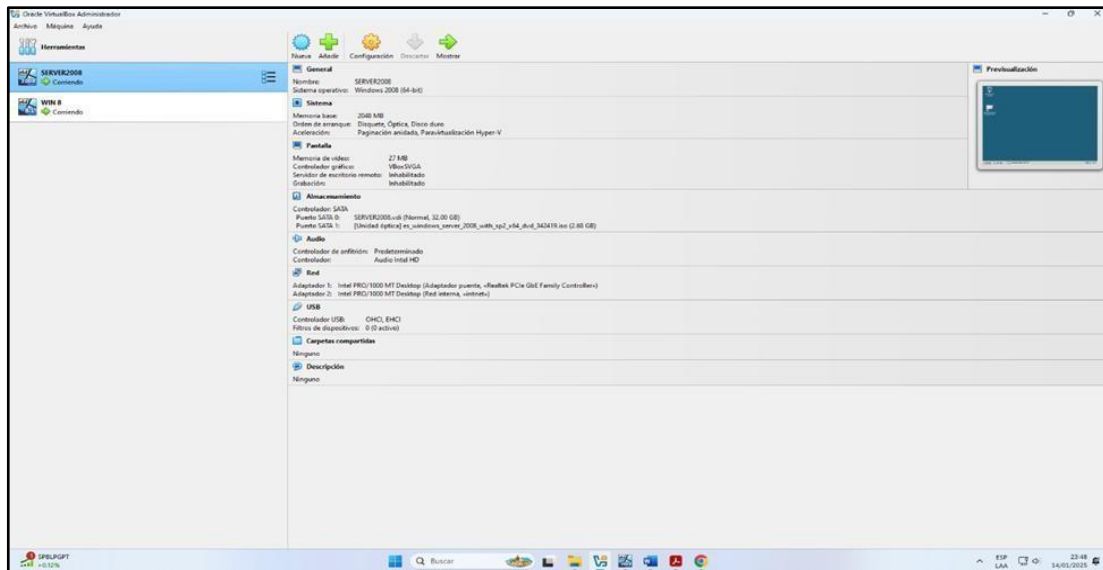
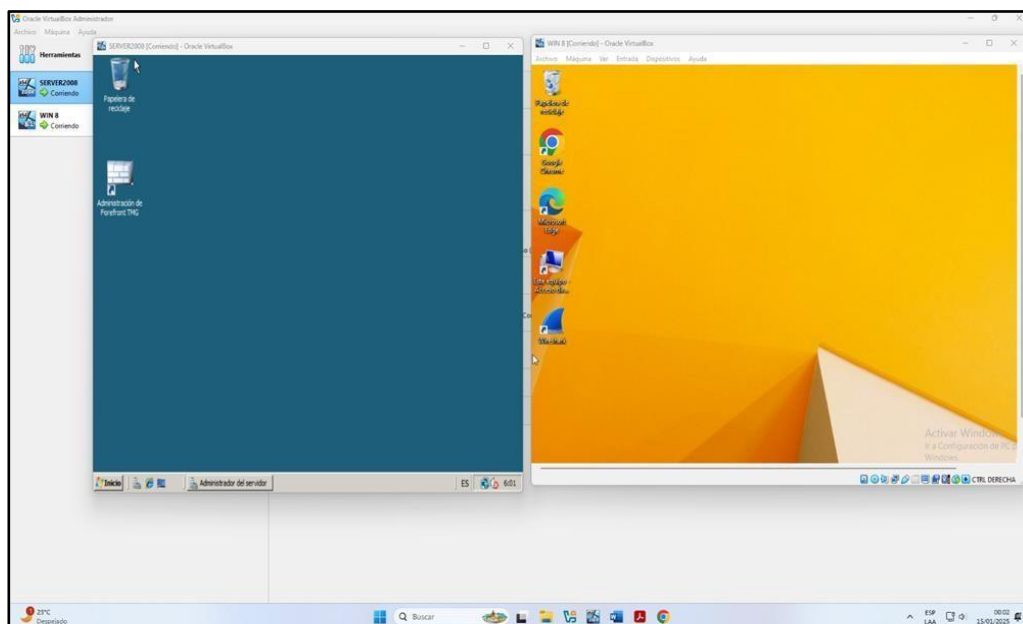


FIGURA 27

Configuración de los equipos.



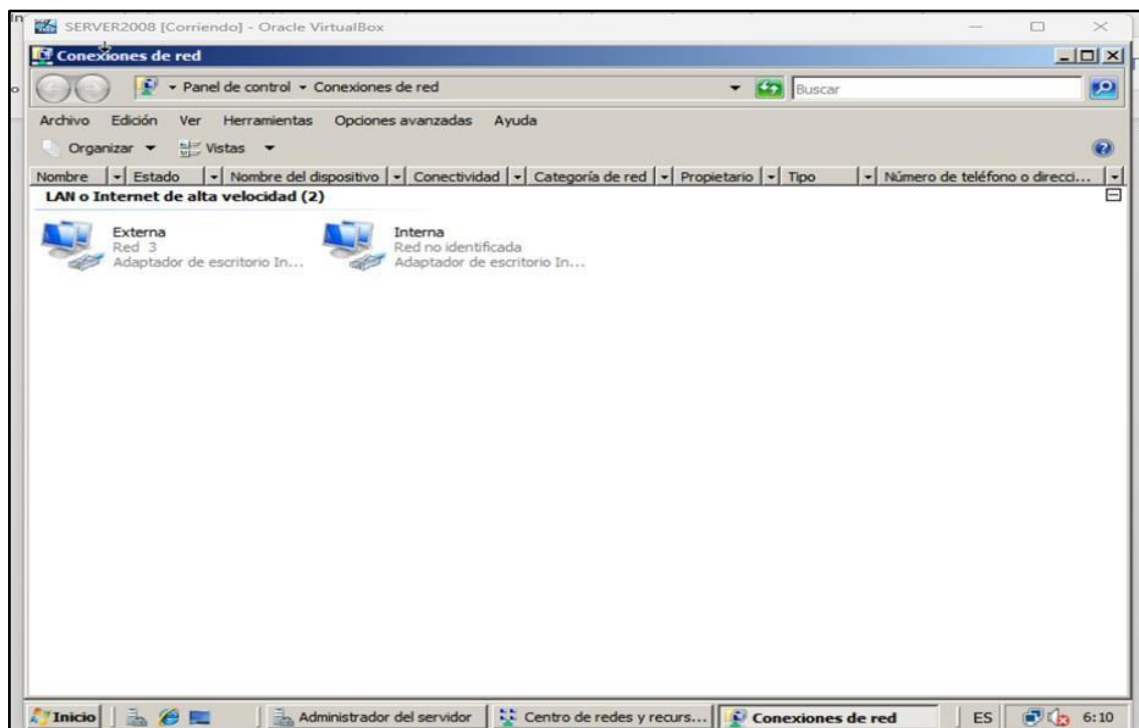
Configuración de la Red

Para establecer las direcciones IP en VirtualBox, es necesario incorporar un adaptador de red extra, al cual se le denominará Externo. Después, se asignarán las direcciones IP siguientes:

- **IP Externa:** 192.168.0.106
- **IP INTERNA:** 10.10.10.1 (nuestra red LAN usa esta nomenclatura en adelante).

FIGURA 28

Configuración de red LAN.



Instalación de ISA Server

La instalación de ISA Server 2010 en el servidor se realizará a continuación.

ISA Server es un cortafuegos sofisticado que ofrece protección frente a intrusos y facilita la administración de normas de seguridad para la red perimetral. En el proceso de instalación, el asistente orienta el procedimiento paso a paso.

FIGURA 29

Instalación de Isaserver.

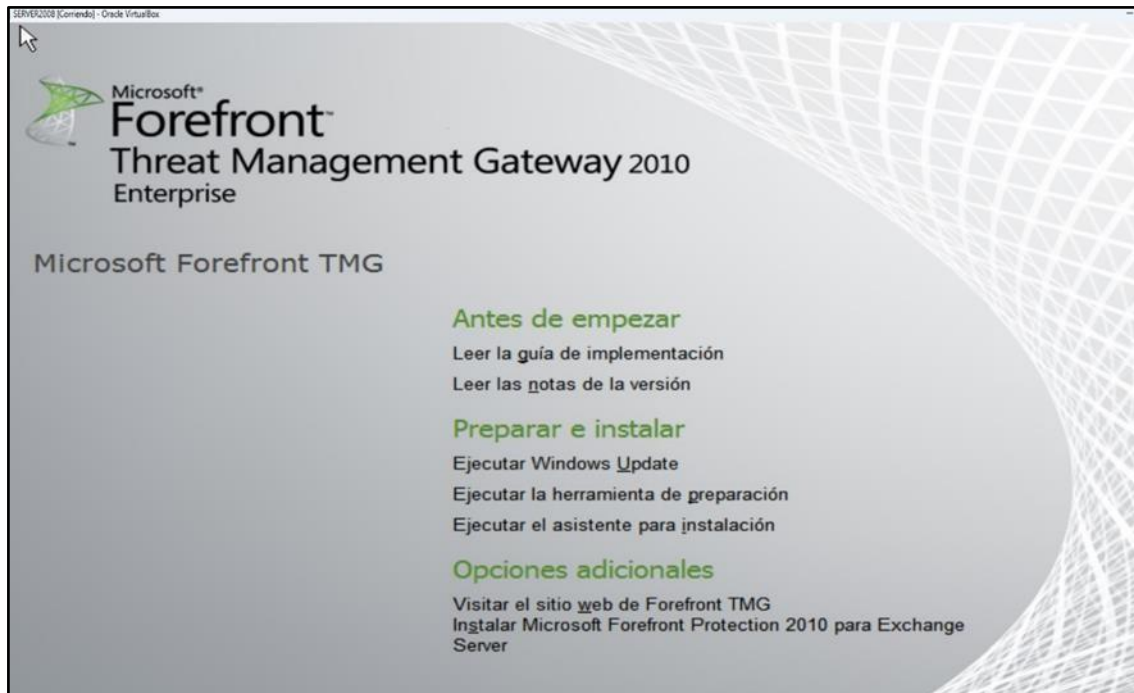
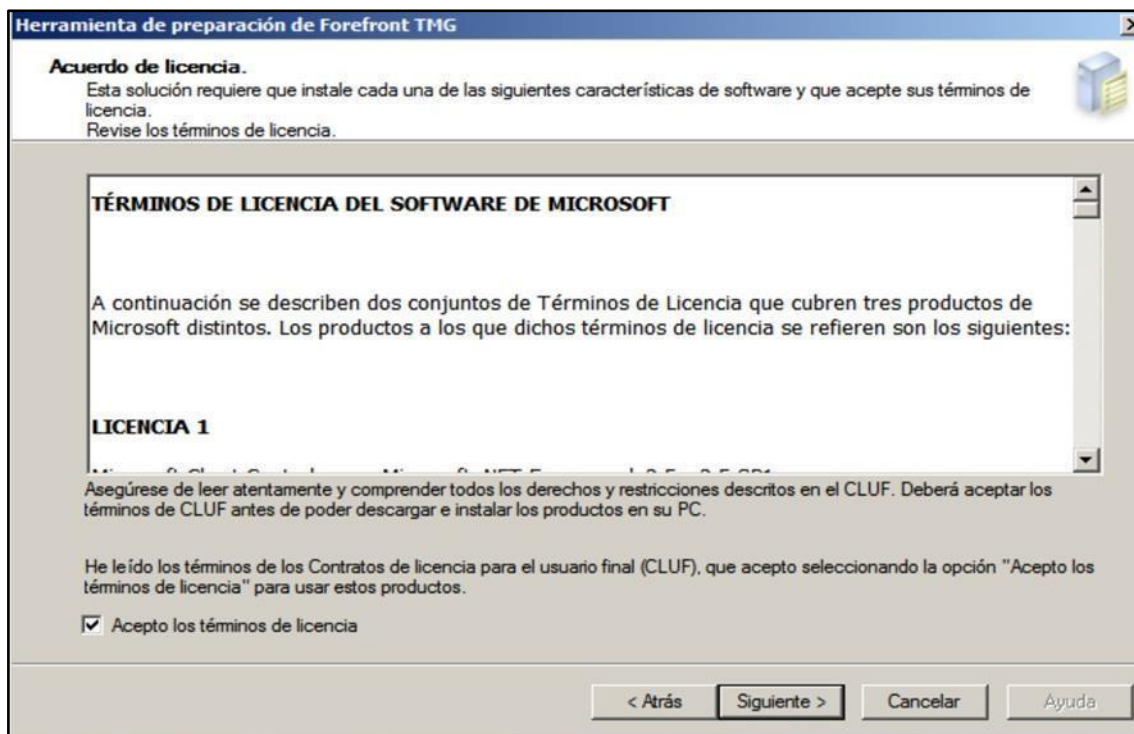


FIGURA 30

Términos y condiciones de Isa Server.



Una vez instalado, ISA Server funciona a través de la generación de normas que establecen las políticas de seguridad. Su configuración posibilita gestionar la conexión a Internet de los dispositivos dentro de la red, funcionando de la siguiente manera:

FIGURA 31

Instalación completada.

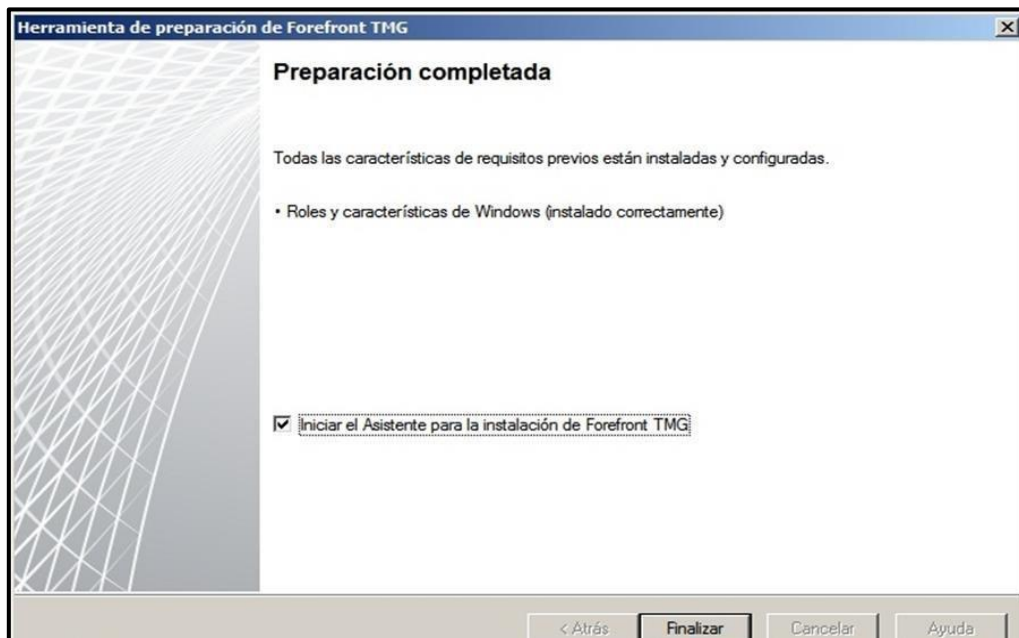


FIGURA 32

Consola Forefront TMG.



Configuración de Directivas de Firewall

Al finalizar la instalación, ISA Server presenta tres funciones principales que pueden configurarse según las necesidades de la infraestructura.

FIGURA 33

Configuración de Directiva de Firewall.

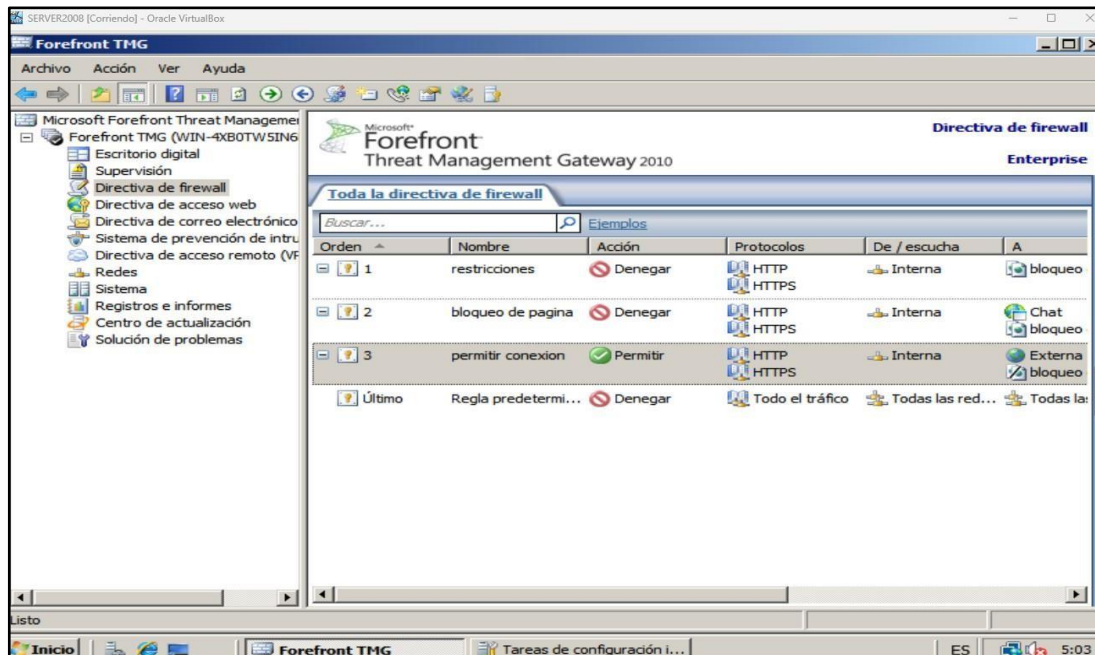
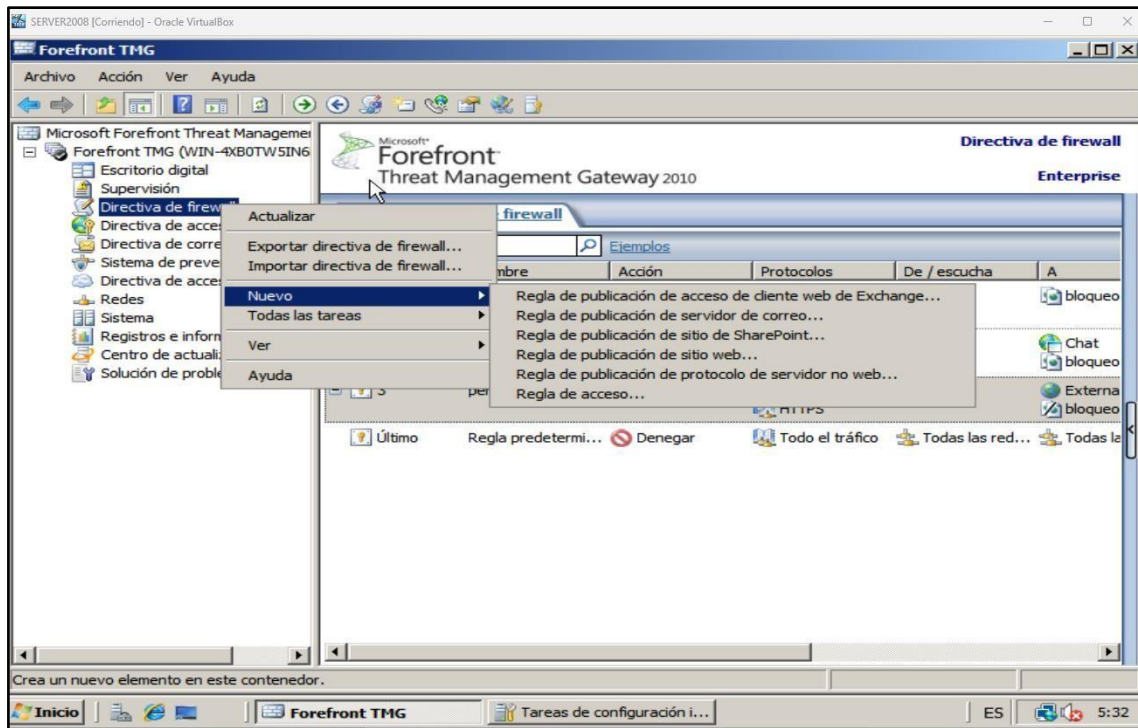


FIGURA 34

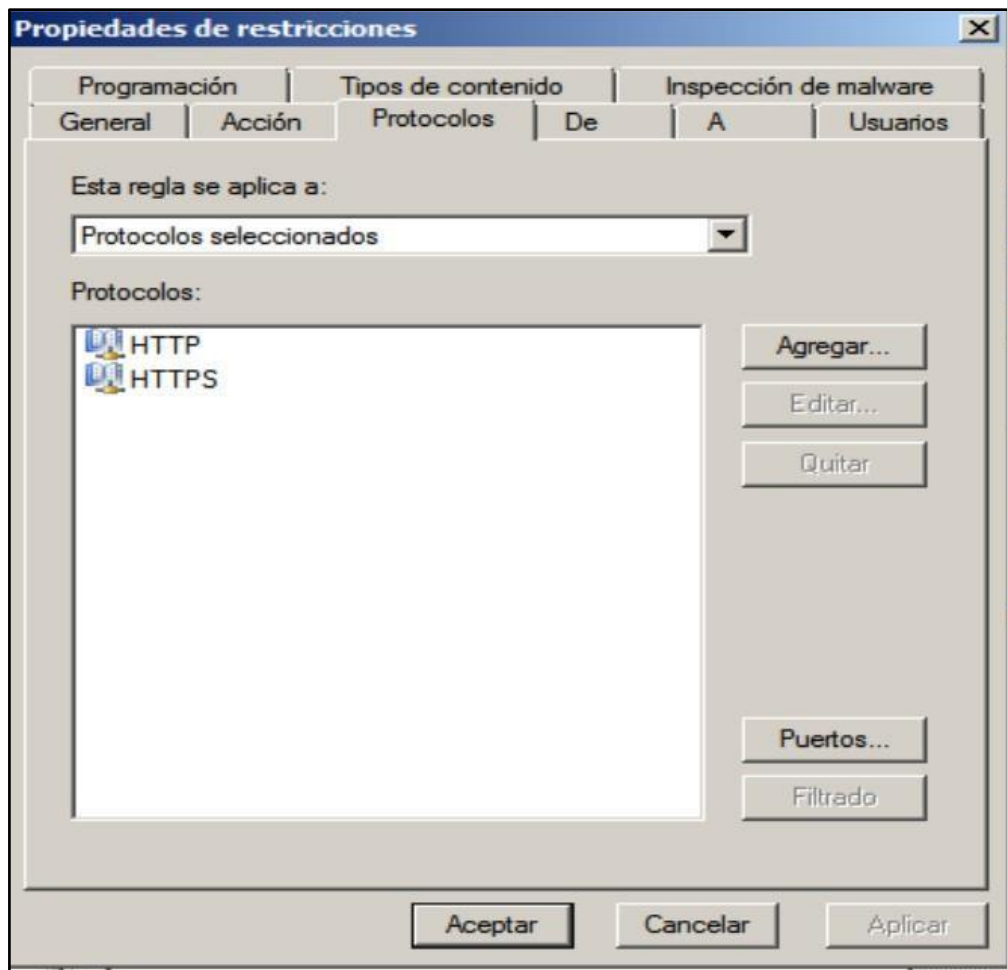
Creación de nueva regla de acceso



Para establecer una regla de acceso, es necesario agregar los **protocolos permitidos**.

FIGURA 35

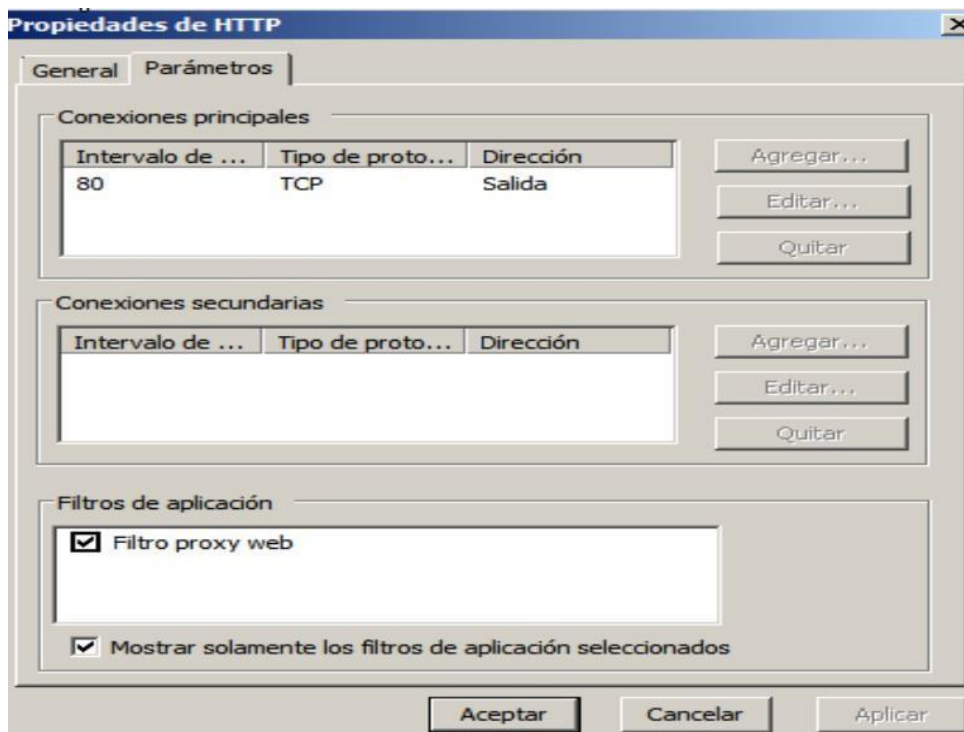
Agregar protocolos



Es posible acceder a las características de HTTP y añadir el puerto 80 al protocolo TCP.

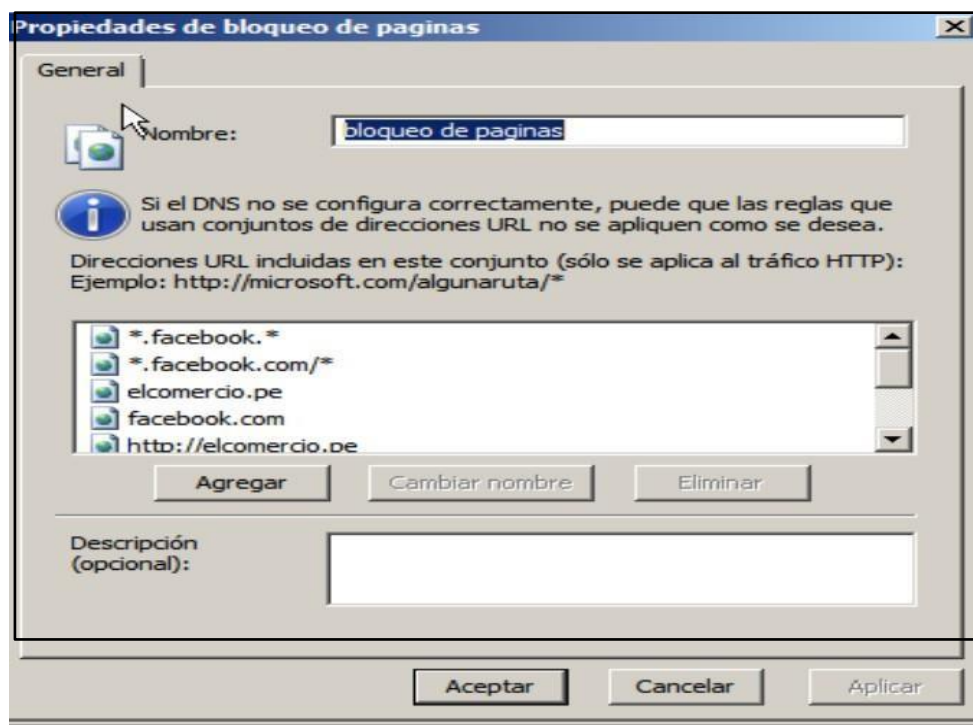
Luego, se pone en marcha el filtro proxy web, que permite redirigir el tráfico por medio del proxy para distinguir los servicios web del grupo como tráfico de red.

FIGURA 36
Propiedades de HTTP.



Agregamos las URL a los que los usuarios no podrán ingresar.

FIGURA 37
Agregar paginas a bloquear.



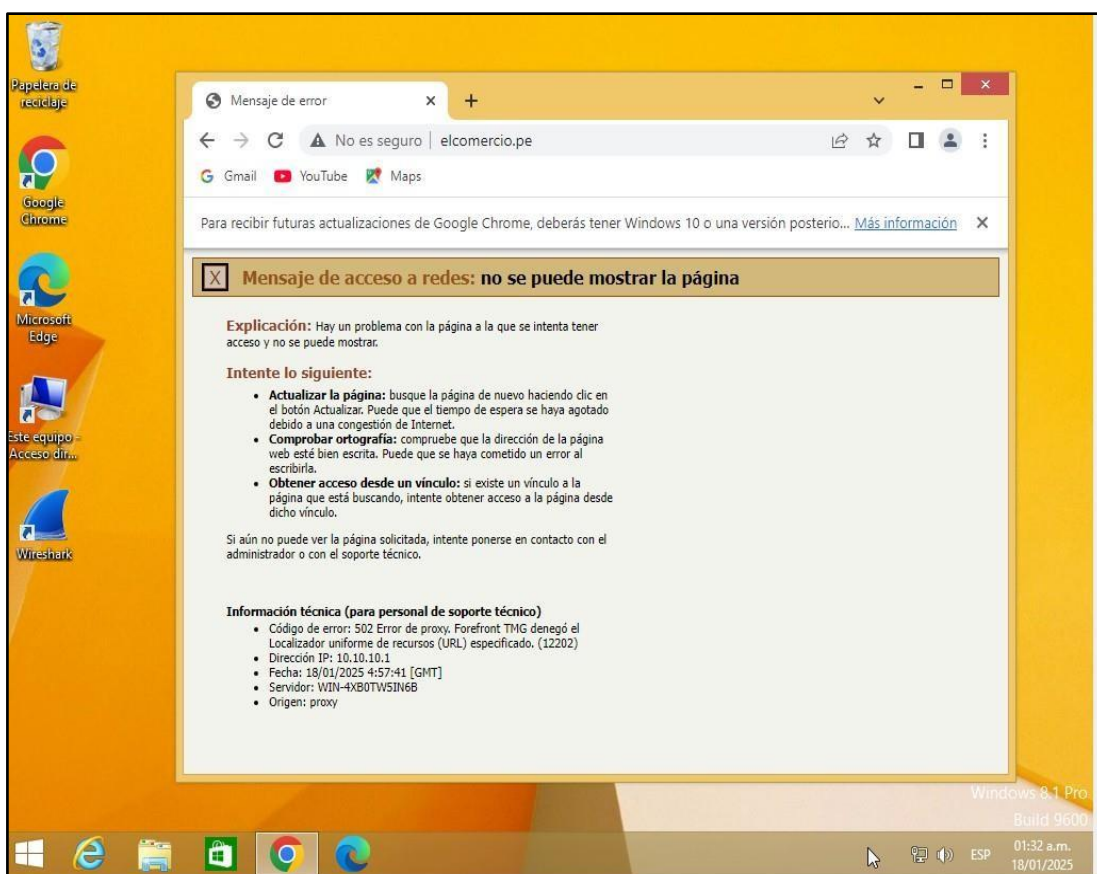
En nuestra situación, hemos puesto algunas páginas para las restricciones, luego aceptamos y conservamos las modificaciones.

Validación del Bloqueo de Páginas

Para verificar la implementación adecuada de las normas, se entra en un equipo cliente con Windows 8 y se trata de acceder a las páginas que previamente se han bloqueado.

FIGURA 38

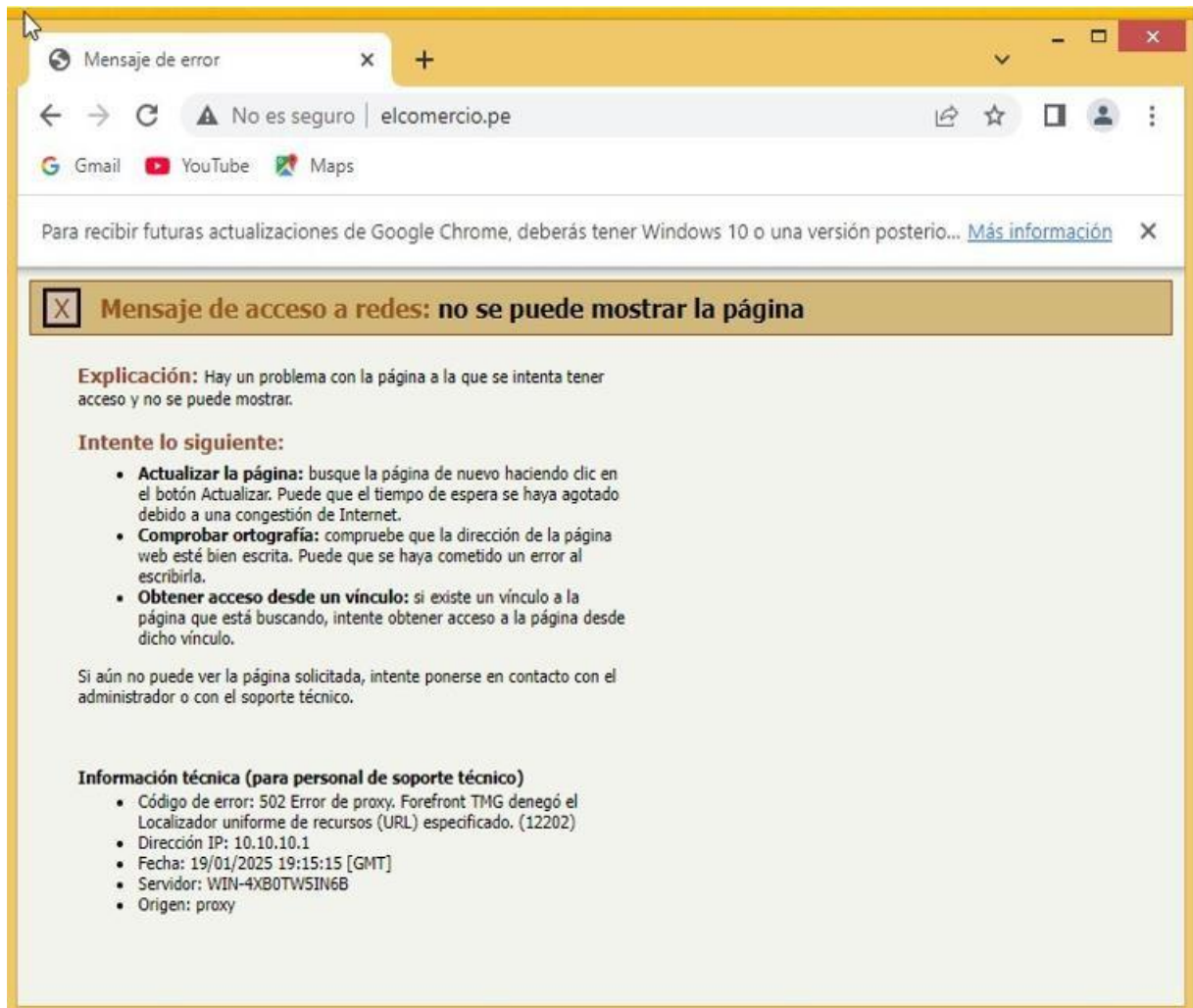
Página bloqueada.



En la figura 38, verificamos ingresando a la página de ejemplo www.elcomercio.pe y validamos que la página está siendo bloqueada con la información mencionada en la parte inferior, haciendo referencia que si se cumple la regla implementada desde el servidor.

FIGURA 39

Referencia de página informativa bloqueada.



En la figura 39, el sistema genera un mensaje de bloqueo a través del Servidor TMG, confirmando la correcta aplicación de la regla implementada.

Software y descarga para instalación

- **Virtual Box:** <https://www.virtualbox.org/wiki/Downloads>
- **Windows** **Server** **2008:** <https://www.microsoft.com/es-es/download/details.aspx?id=24212>
- **Windows 8:** <https://www.microsoft.com/es-es/software-download>
- **Wireshark:** <https://www.wireshark.org/download.html>

- IIS Server 2010: [https://learn.microsoft.com/es-es/previous-versions/tn-archive/cc441520\(v=technet.10\)](https://learn.microsoft.com/es-es/previous-versions/tn-archive/cc441520(v=technet.10))
- Jupyter notebook: <https://docs.jupyter.org/en/stable/install.html>